
KINETIC-MAMBA: MAMBA-ASSISTED PREDICTIONS OF STIFF CHEMICAL KINETICS

Additi Pandey^{a,*}, Liang Wei^b, Hessam Babae^c, George Em Karniadakis^a

^a Division of Applied Mathematics, Brown University, Providence, RI, USA

^b Karagozian & Case, Inc., Glendale, CA, USA

^c Department of Mechanical Engineering and Materials Science, University of Pittsburgh, Pittsburgh, PA, USA

ABSTRACT

Accurate chemical kinetics modeling is essential for combustion simulations, as it governs the evolution of complex reaction pathways and thermochemical states. In this work, we introduce Kinetic-Mamba, a Mamba-based neural operator framework that integrates the expressive power of neural operators with the efficient temporal modeling capabilities of Mamba architectures. The framework comprises three complementary models: (i) a standalone Mamba model that predicts the time evolution of thermochemical state variables from given initial conditions; (ii) a constrained Mamba model that enforces mass conservation while learning the state dynamics; and (iii) a regime-informed architecture employing two standalone Mamba models to capture dynamics across temperature-dependent regimes. We additionally develop a latent Kinetic-Mamba variant that evolves dynamics in a reduced latent space and reconstructs the full state on the physical manifold. The accuracy and robustness of Kinetic-Mamba was evaluated using both time-decomposition and recursive-prediction strategies. We further assess the extrapolation capabilities of the model on varied out-of-distribution datasets. Computational experiments on Syngas and GRI-Mech 3.0 reaction mechanisms demonstrate that our framework achieves high fidelity in predicting complex kinetic behavior using only the initial conditions of the state variables.

Keywords State-space Models · Mamba · Digital Twin · Stiff Differential Equations · Chemical Kinetics · Dimensionality Reduction · Principal Component Analysis

1 Introduction

Modeling chemically reacting flows requires solving detailed chemical kinetic mechanisms that may involve dozens to thousands of species and reactions. The governing system of ordinary differential equations (ODEs) is typically highly stiff, characterized by the coexistence of vastly disparate time scales, from fast radical chain-branching reactions occurring on microsecond or nanosecond scales to slow recombination or diffusion-limited processes evolving over milliseconds or longer.

The stiffness in the system of ODEs introduces severe numerical difficulties. Explicit time-integrators require the timestep to match the fastest chemical timescale, making them prohibitively expensive, even though the physics of interest may evolve on far slower scales. Implicit methods, which are generally A-stable, are therefore widely used to solve stiff chemical

* Corresponding author: additi_pandey@brown.edu (Additi Pandey)

kinetics. However, they require repeated Jacobian evaluations and nonlinear iterations, leading to very high computational cost [1, 2]. Stiff ODE systems remain challenging even with advanced implicit techniques. Several methods have been developed to alleviate the algebraic burden while maintaining A-stability [3–6], yet these approaches are often limited in their ability to extrapolate solutions over long physical times. Linearly implicit Rosenbrock methods require solving only a linear system, significantly reducing the overhead in solving non-linear equations from the implicit solvers [7]. Nevertheless, for reaction mechanisms involving hundreds and thousands of species, the resulting linear systems can still be extremely large, and solving them remains a major bottleneck.

Despite decades of advances in stiff ODE solvers, the efficient integration of detailed chemical kinetics remains a major challenge, particularly in turbulent reacting-flow simulations in computational fluid dynamics (CFD). This strongly motivates the development of innovative acceleration strategies, including machine-learning-based surrogate models, to reduce the computational overhead associated with stiff chemical source-term evaluations while retaining physical fidelity. In this regard, several machine learning models have been proposed such as artificial neural networks (ANNs)-based models [8] and parallel ResNets [9]. Physics-informed neural networks (PINNs) proposed by [10] have also been used to accelerate stiff chemistry via quasi-steady state assumptions [11] or through extreme theory of functional connections [12]. Neural operators such as DeepONets [13], FNOs [14] and DeepOKANs [15, 16] have also been widely used to accelerate stiff chemistry as the models can be trained offline, which takes the bulk of computational cost in consideration, and deployed online [17–21]. Most of these existing approaches, however, either rely on assigning a separate network to each variable or introducing a pre-transformation network within their model architecture, or depend on adaptive weighting schemes to handle variables and samples yielding higher errors.

Since the equations that govern the dynamics of state variables are time-dependent ODEs, we can treat the trajectory of each state variable as a sequence in time. With the advent of transformer-based models, sequence modeling has been playing a major role in deep learning. However, the primary downside of employing transformers for tasks such as predicting the chemical kinetics of several state variables leads to substantial computational overhead due to the quadratic scaling with the sequence length. Gu et al. [22] introduced structured state space models (SSMs), which are foundational models of control theory. Through efficient reparameterization techniques introduced by Gu et al., structured state space sequence (S4) models have linear or near-linear scaling with sequence length to capture long dependencies, but showed promising results mostly on continuous data. Consequently, selective SSMs were introduced to model dynamical systems and learn long-range dependencies with linear sequence length. They employ a selection mechanism to parameterize the SSM parameters on the basis of the input. Combining the SSM architecture with the Multilayer Perceptron (MLP) transformer block produces Mamba [23] introduced by Gu and Dao in 2023. Hu et al. [24] has already touched on several aspects of modeling dynamical systems with Mamba, showcasing its efficiency and promising accuracy. While they demonstrate Mamba’s ability to capture mildly stiff ODEs, the stiff chemical kinetics considered in our work, pose significant challenges that have not been addressed.

Along with the promising applications of Mamba as language model, and its growing success in modeling dynamical systems, its usage in chemical and biological sciences is also increasing rapidly. Brazil et al. [25] introduced a Mamba-based chemical foundation model trained on molecular data obtained from PubChem and demonstrated its efficient performance on varied tasks, including molecular property classification and prediction and prediction of chemical reaction yields. Lombard et al. [26] explored the effectiveness of Mamba for molecular generation. Xu et al. [27] introduced SMILES-Mamba that leverages a combination of self-supervised pretraining on SMILES strings and fine tuning strategies to effectively capture the chemical structures as well as chemical relationships present within the molecular data. Xu et al. [28] proposed a Mamba-based architecture for protein function prediction by pretraining the model on protein amino acid sequence and fine-tuning it on specific protein functions. While these works demonstrate Mamba’s capacity in dealing with chemical data, they do not address time-dependent predictions of stiff dynamical systems such as in chemical kinetics.

In this work, we extend the capabilities of Mamba-based models by applying them to accurately predict the dynamics of stiff chemical systems. We introduce Kinetic-Mamba framework that achieves comparable or better accuracy on similar mechanisms using single step training, without separate networks for each variable or adaptive weighting of the loss function.

1.1 Contributions

We present our framework Kinetic-Mamba (KM), which predicts the dynamics of the state variables involved in the reaction from the given conditions of state variables at a specific time point. Drawing on the approach of Nath et al. [21], our model leverages time decomposition, extrapolation as well as recursive prediction to learn this mapping. Through this work, we want to demonstrate the effectiveness of selective SSMs like Mamba [23] in successfully modeling the stiff differential equations governing complex chemical kinetics.

In this study, we present the following novelties:

1. We present Kinetic-Mamba (KM), a Mamba-based framework for predicting chemistry in stiff systems. Our Kinetic-Mamba framework consists of four different models:
 - (a) A single Mamba model that predicts the dynamics of all state variables from the given initial conditions.
 - (b) Two Mamba models that work together to address different ignition regimes present due to different initial conditions for temperature.
 - (c) A latent Mamba model with asymmetric input-output design, where the model receives d -dimensional projections as input and predicts directly in the full $(m + 1)$ -dimensional state space, where $d < (m + 1)$.
 - (d) A mass-conserving Kinetic-Mamba (KM) model to automatically enforce the constraint that the sum of mass fractions of all the species equals to unity.
2. Our Kinetic-Mamba model is able to predict the dynamics of state variables on variable length time windows irrespective of being trained on windows of fixed temporal length, enabling direct integration with adaptive CFD time-stepping schemes.
3. We demonstrate the accuracy of our Kinetic-Mamba (KM) framework not only on the dataset generated using parameters sampled from a uniform distribution but also on the dynamics of state variables generated using pressure, temperature and equivalence ratios sampled from a random distribution. This allows us to perform a more realistic and holistic assessment of the prediction accuracy of our proposed framework.

2 Methodology

Mamba is an extension of the structured state-space model (S4) [23]. These models utilize the following equations to map input sequence $x(t)$ into the output sequence $y(t)$ via hidden state $h(t)$:

$$\begin{aligned} (1a) \quad & h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \\ (1b) \quad & y(t) = \mathbf{C}h(t). \end{aligned}$$

For an input $x \in \mathbb{R}^{\text{input}}$, the matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are in $\mathbb{R}^{\text{hidden} \times \text{hidden}}, \mathbb{R}^{\text{hidden} \times \text{input}}, \mathbb{R}^{\text{output} \times \text{hidden}}$, respectively, where $h \in \mathbb{R}^{\text{hidden}}$ and $y \in \mathbb{R}^{\text{output}}$ [23, 29]. For problems involving time discretization using the continuous time sequence to sequence mapping is not ideal, hence we discretize the above equations using zero-order hold:

$$\begin{aligned} (2a) \quad & h_t = \bar{\mathbf{A}}h_{t-1} + \bar{\mathbf{B}}x_t, \\ (2b) \quad & y_t = \mathbf{C}h_t, \end{aligned}$$

such that $\bar{\mathbf{A}} = f(\Delta, \mathbf{A}) = \exp(\Delta\mathbf{A})$ and $\bar{\mathbf{B}} = f(\Delta, \mathbf{A}, \mathbf{B}) = \exp(\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot (\Delta\mathbf{B})$, where $\Delta, \mathbf{A}, \mathbf{B}$ are the continuous parameters. The continuous and the discrete parameters are linear

time invariant, that is, they are fixed for all time steps. Mamba overcomes this by introducing a selection mechanism, by making $\mathbf{B}$, $\mathbf{C}$ and Δ as functions of the current input. Combined with hardware-aware algorithms (involving kernel fusion, parallel scan algorithm, and recomputation to reduce memory requirements), Mamba offers linear time efficiency, making it adept in sequence modeling. For stiff problems specifically, the discrete formulation closely resembles exponential integrators, which along with the dependency on current inputs, makes Mamba well-suited for stiff problems.

In this study, we consider a deep sequence backbone of repeating Mamba-1 [23] blocks with linear input and output projection. We shall refer to it as a standalone Mamba model. To elucidate the model, consider that the input is given by $\mathbf{X} \in \mathbb{R}^{1 \times (k+1) \times p}$ which has initial conditions known at t_0 tiled across all time steps over which prediction is to be made. Although the initial condition is tiled across all time steps, temporal evolution is governed internally by the selective SSM blocks. We first project $p \rightarrow N$, where N is the hidden dimension and process our input through L blocks consisting of standard residual and normalization layers, followed by feeding the resulting state $\mathbf{X}'$ to the Mamba model. The output of the Mamba model $\mathbf{Y}'$ is processed through residual and normalization layers along with the application of MLP. After the final block, a final addition of residual and normalization is applied, after which the output is projected $N \rightarrow p$ dimensions yielding $\mathbf{Y}$. Inside a single Mamba block as shown in Fig. 1, there is first a linear projection that projects the N -dimensional hidden space onto a higher dimensional latent space. The input is split along the feature dimension. A 1-dimensional convolution is applied over the temporal direction followed by a non-linear activation function to the first part. The output is then fed into the selective SSM block where it is evolved in time via SSM equations using parallel scan. On the second part, a non-linear activation function is applied and it is then combined with the output of SSM block via multiplication. The resultant output is finally projected back onto N -dimensional space.

The objective of this study is to provide an accurate prediction of the dynamics of the different state variables involved in a chemical reaction, governed by a system of stiff ordinary differential equations, given the initial conditions of these state variables. For a chemical reaction involving m species, the state vector consists of $m + 2$ state variables, making $p = m + 2$, accounting for temperature as well as pressure or density that influence the reaction. This information depends on whether the reactor assumes constant volume or constant pressure.

2.1 Normalization Strategy and Input Construction

Suppose that our dataset consists of N training samples, M testing samples, $n_t = 10,000$ total time steps, and $m + 2$ thermodynamic state variables. Moreover, suppose $\Delta t = 10^{-7}$ seconds. We denote the training and testing datasets with $X_{tr} \in \mathbb{R}^{N \times n_t \times (m+2)}$ and $X_{te} \in \mathbb{R}^{M \times n_t \times (m+2)}$, respectively. Since the dataset has been generated using Cantera [30] solver, it may result in some extremely small negative values in the species mass fractions due to numerical error. We therefore, substitute any negative value with 0. Thus, our dataset lies in $\mathbb{R}_{\geq 0}^{s \times n_t \times (m+2)}$, where $s \in \{N, M\}$ depending on training or testing dataset.

Since in a CFD-integrated setting, the solver provides initial condition at regular intervals, we emulate this by assuming that within an interval of $100\Delta t$ seconds, we know the initial conditions coming from the computational fluid dynamics (CFD) solver. However, since the integration of the framework with a CFD solver is beyond the scope of this study, we assume that the initial conditions are known at every $10 \mu s$ or $k = 101$ st time point. We then construct our model input by performing time decomposition. To do so, we convert our dataset of N (or M) samples, n_t time steps, and $m + 2$ state variables into dataset consisting of $N * 99$ (or $M * 99$) samples, 101 time steps and $m + 2$ state variables. We achieve this by segmenting our time domain into 99 segments of length 101 where the 101st time point overlaps with the first time point of next segment. We then translate all segments within $[0, 100]$ time step range. The time decomposition scheme can be thought of as a mapping from $\mathbb{R}_{\geq 0}^{s \times n_t \times (m+2)} \rightarrow \mathbb{R}_{\geq 0}^{(s*99) \times 101 \times (m+2)}$ which transforms both X_{tr} and X_{te} , where $s \in \{N, M\}$ depending on training or testing dataset, respectively. The idea is to be able to learn the dynamics within small temporal windows

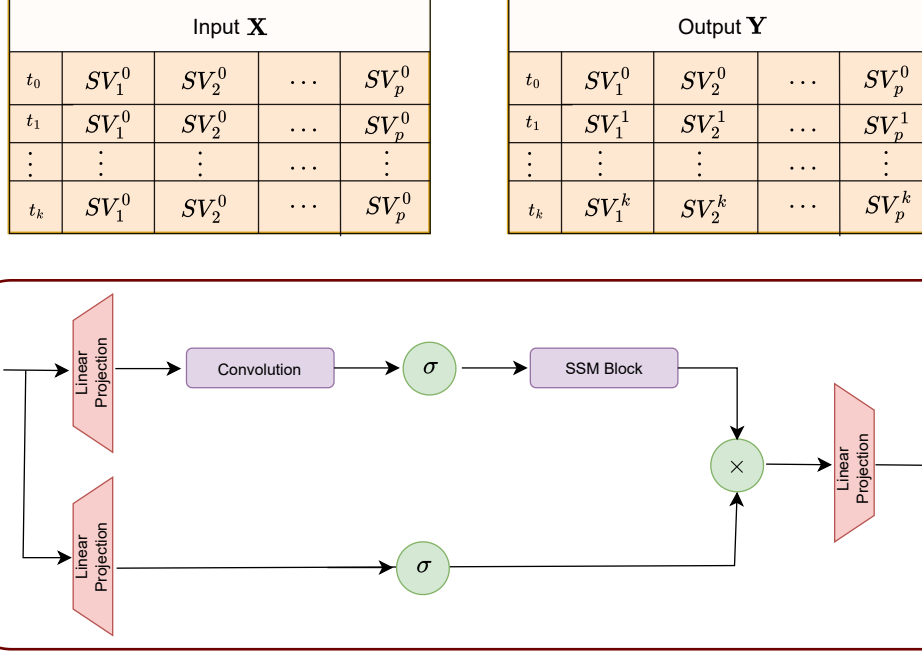


Fig. 1: **Schematic representation of the Mamba block [23]:** The entire Mamba block consists of SSM related block and a skip connection block, combined via element-wise multiplication. The SSM related block consists of linear projection, convolution, non-linear activation function (SiLU) and SSM block, while the skip connection block consists of a linear transformation followed by non-linear activation. In the figure, the input $\mathbf{X}$ consists of initial values of state variables tiled to unknown time points, and $\mathbf{Y}$ has predicted values of state variables at unknown time points. These are denoted in the form of SV_a^b , where a is the type of state variable (temperature, density, CO mass fraction, HO mass fraction, etc) and b is the time. Moreover, $\mathbf{X}'$, $\mathbf{Y}'$ represent $\mathbf{X}$, $\mathbf{Y}$ in N -dimensional hidden space after passing through the standard residual and norm layers.

and capture any stiffness locally that occurs in the dynamics of the state variables. After time decomposition on our dataset, we proceed to normalize it.

Different state variables involved in a chemical reaction have different magnitudes that vary widely from each-other. Due to this, we need to scale these values within a common range, say $[-1, 1]$, in order to enhance the convergence of our Kinetic-Mamba framework, without leaving smaller species unresolved. Consider $X \in \mathbb{R}_{\geq 0}^{(s*99) \times 101 \times (m+2)}$, where s are the number of samples given by N or M , depending on whether it corresponds to the training dataset or testing dataset. We first perform a power transformation given by $\tilde{X} = X^{1/5}$ inspired by [19], compressing the dynamic range, while also accounting for zeros in the dataset. Corresponding to t_0 or the first time point, we obtain $\tilde{X}^0 \in \mathbb{R}_{\geq 0}^{(s*99) \times (m+2)}$, which is the number of time-decomposed samples for different state variables at the given initial time. We find the minimum and maximum values from the training dataset $\tilde{X}_{tr}$ and $\tilde{X}_{tr}^0$ to perform min-max normalization over our training and testing datasets as well as their corresponding initial condition inputs. Using the following mapping corresponding to $\tilde{X}_{tr}$ and $\tilde{X}_{tr}^0$, respectively, we map our dataset to a compact interval $[-1, 1]$:

$$(3) \quad \mathcal{N}(\tilde{X}) = 2 \left(\frac{\tilde{X} - (\tilde{X}_{tr})_{min}}{(\tilde{X}_{tr})_{max} - (\tilde{X}_{tr})_{min}} \right) - 1, \text{ where 'tr' corresponds to the training dataset}$$

$$(4) \quad \mathcal{N}(\tilde{X}^0) = 2 \left(\frac{\tilde{X}^0 - (\tilde{X}_{tr}^0)_{min}}{(\tilde{X}_{tr}^0)_{max} - (\tilde{X}_{tr}^0)_{min}} \right) - 1, \text{ where 0 corresponds to the time step } t_0$$

The minimum and maximum values have been computed for each state variable across all time steps for all time decomposed samples of the training dataset. Moreover, for the initial conditions of the training dataset, the minimum and maximum values have been computed along the sample direction. Once the datasets have been normalized, we construct the input (which consists of the initial conditions or data corresponding to t_0) on a grid. To do this, we construct a grid of $k = 101$ time points, corresponding to $s * 99$ samples, where $s \in \{N, M\}$ depending on whether it is training or testing data, for all state variables. Since we only know input values corresponding to t_0 for different samples, we tile these values to the remaining time points, where the dynamics are unknown. The reason for this is to make the model aware of the time points where dynamics is to be predicted, while avoiding introducing any bias, as 0 is a valid normalized value in the normalized dataset.

2.2 Kinetic-Mamba Models

In this section, we focus on the discussion of the various models that make up the Kinetic-Mamba framework. In this study, we shall refer to our principal model as the standalone Mamba model that consists of a deep sequence backbone of repeating Mamba blocks, with linear input and output projection. A similar model has been employed before by [Hu et al.](#) in [24]. The input to the model consists of tensors of shape $(s * 99, 101, m + 2)$, where $s \in \{N, M\}$ denoting the training or testing samples, respectively, constructed by taking the initial condition at t_0 and tiling it across the remaining 100 time steps as discussed in Section 2.1.

When the dataset has all profiles for each variable exhibiting similar nature of evolution, one model can efficiently learn the dynamics. However, sometimes the evolution of temperature with time shows distinct ignition regimes in its dynamics. This could be due to the system being weakly reactive with negligible heat release for a specific temperature range, while beyond the range, the chain-branching reactions may intensify rapidly, producing a rapid temperature rise and a new steady state. In this case, there is a critical ignition threshold at a certain τ K. For the dataset considered in this study, we find the value of τ by observing that the profiles that are below the threshold τ K, satisfy the following equation: Given $\epsilon > 0$:

$$(5) \quad \tau = \max_{i \in S_\epsilon} T_i(0), \quad S_\epsilon = \left\{ i \mid \frac{T_i(n_t) - T_i(1)}{n_t - 1} < \epsilon, i = 0, 1, \dots, N_s \right\}$$

where N_s is the total number of samples, including both the training and testing datasets. We then partition our dataset into two subsets such that initial temperature values corresponding to values below and equal to τ form non-igniting regime and the remaining values form igniting regime. For each of these subsets, we create the corresponding training and testing datasets and then employ an independent standalone Mamba model on each of them. We refer to this setup as ignition regime-informed KM model.

Inspired by [21], we also introduce a mass-conserving Mamba model within the Kinetic-Mamba framework. The mass fractions of species is defined as the mass of the species divided by the total mass of all the species involved in the reaction. Hence, the sum of mass fractions of all the species should be equal to 1. Since our predictions should at any instant of time satisfy this constraint, we implement the mass-conserving model within our Kinetic-Mamba framework. The mass-conserving model consists of a standalone Mamba model which automatically enforces the conservation of mass constraint. The conservation of mass constraint, which in this case, corresponds to the sum of all mass fractions equal to unity, is enforced automatically, by noting that the conservation of mass criterion is a linear constraint on the mass fraction. Given m species, we could encode the mass fraction vector using a $(m - 1)$ -dimensional vector. Inspired by [31], in m dimensions, we define the forward mapping by:

$$(6) \quad z_1, z_2, \dots, z_{m-1} = g(y_1, y_2, \dots, y_m)$$

We call this function g , where $g : [-1, 1]^m \rightarrow [0, 1]^{m-1}$, such that $g(\mathbf{y}) \mapsto g(\mathbf{z})$. This function g is invertible and therefore, is bijective. Moreover, we have:

$$(7a) \quad z_k = \frac{y_k}{1 - \sum_{j=1, j \neq k}^{m-1} y_j}, \quad 1 \leq k \leq m-2$$

$$(7b) \quad z_{m-1} = y_{m-1}$$

To find the inverse map $(g^{-1}(\cdot))$, such that

$$(8) \quad y_1, y_2, \dots, y_m = g^{-1}(z_1, z_2, \dots, z_{m-1})$$

We start from the forward map

$$(9) \quad z_k = \frac{y_k}{d_k}, \quad k = 1, \dots, m-2, \quad \text{where} \quad d_k = 1 - \sum_{j=1, j \neq k}^{m-1} y_j,$$

Using a simple substitution on y_k , we obtain a system of linear equations given by:

$$(10) \quad d_k + \sum_{j \neq k}^d z_j d_j = 1, \quad \text{for } k = 1, \dots, m-2.$$

which for simplicity can be rewritten as:

$$(11) \quad \mathbf{A} \mathbf{d} = \mathbf{b},$$

where: $\mathbf{d} = [d_1, d_2, \dots, d_{m-2}]^T$ is the vector of unknowns, $\mathbf{b} = [1, 1, \dots, 1]^T$ is a vector of ones, and $\mathbf{A}$ is a $(m-2) \times (m-2)$ coefficient matrix defined as:

$$(12) \quad A_{kj} = \begin{cases} 1, & \text{if } k = j, \\ z_j, & \text{if } k \neq j. \end{cases}$$

Once the linear system is solved, we can compute the original m -dimensional mass fraction vector as:

$$(13a) \quad y_k = z_k d_k, \quad 1 \leq k \leq m-2$$

$$(13b) \quad y_{m-1} = z_{m-1}$$

$$(13c) \quad y_m = 1 - \sum_{k=1}^{m-1} y_k.$$

A detailed account, including the derivation of the same, can be found in [21]. For our purpose, this implies that instead of training the model for m species, density or pressure and temperature, we just need to train it for $(m+1)$ variables as our total number of variables reduces by a count of 1.

2.2.1 Latent Kinetic-Mamba model

In realistic scenarios of complex reactions, there are multiple state variables whose dynamics needs to be taken into account for a successful simulation of chemical kinetics. In such situations, it is computationally expensive to train models for all state variables due to an increased number of trainable parameters. We therefore propose a latent Kinetic-Mamba framework. In the latent KM model, we perform PCA on the centered training and testing dataset by doing eigen-decomposition on the training dataset's covariance matrix; we reduce the feature dimension from $m+1$ to d , such that $d < m+1$. To demonstrate the performance of the latent framework, we only consider $m+1$ state variables instead of $m+2$, that is, we consider temperature and m species. Moreover, our latent Kinetic-Mamba model takes input from the d -dimensional manifold and accurately predicts the dynamics of the state variables in the $(m+1)$ -dimensional manifold, learning both the dynamics as well reconstruction in a single model.

To further elucidate this, consider $\mathbf{X}_{tr}, \mathbf{X}_{te}$ to be our training and testing datasets consisting of $m + 1$ state variables, $M * 99, N * 99$ samples and 101 time points. On normalization, we obtain the initial values as $\mathbf{X}_{tr}^0$ and $\mathbf{X}_{te}^0$ after dropping tilde for brevity. Post normalization, we have matrices of size $(N * 99) \times (m + 1)$ and $(M * 99) \times (m + 1)$, respectively. We define $\mathbf{X}_{tr}^{c,0} = \mathbf{X}_{tr}^0 - \overline{\mathbf{X}_{tr}^0}$, where the mean is taken in the sample direction. We construct the covariance matrix given by

$$\Sigma = \frac{(\mathbf{X}_{tr}^{c,0})^T \mathbf{X}_{tr}^{c,0}}{(N * 99) - 1}$$

We then perform eigen decomposition and obtain eigenvalues and the matrix of eigenvectors $\mathbf{V}$. The obtained matrix $\mathbf{V}$ which is an $(m + 1) \times (m + 1)$ matrix is orthonormal. We reduce its dimensions by truncating it into a $(m + 1) \times d$ matrix $\mathbf{V}_r$. We obtain our principal component from $\mathbf{X}_{tr}^{c,0} \cdot \mathbf{V}_r$. Similarly, we center testing dataset by $\mathbf{X}_{te}^{c,0} = \mathbf{X}_{te}^0 - \overline{\mathbf{X}_{tr}^0}$ and then obtain the principal component from $\mathbf{X}_{te}^{c,0} \cdot \mathbf{V}_r$ and denote them with $\mathbf{PC}_{tr}^0, \mathbf{PC}_{te}^0$, respectively.

To train our Kinetic-Mamba model in latent space, we give as input the normalized latent variables $\mathbf{PC}_{tr}^0$, which lies in d -dimensional manifold after input construction (Section 2.1). The normalization is done with respect to the minimum and maximum values of $\mathbf{PC}_{tr}^0$. We train the model using $\tilde{\mathbf{X}}$ which lies in the $(m + 1)$ -dimensional manifold as output labels. This way, the model not only learns the dynamics in the latent space, but also reconstructs the output from d -dimensional manifold to the original $(m + 1)$ -dimensional manifold. Fig. 2 gives an overview of this methodology.

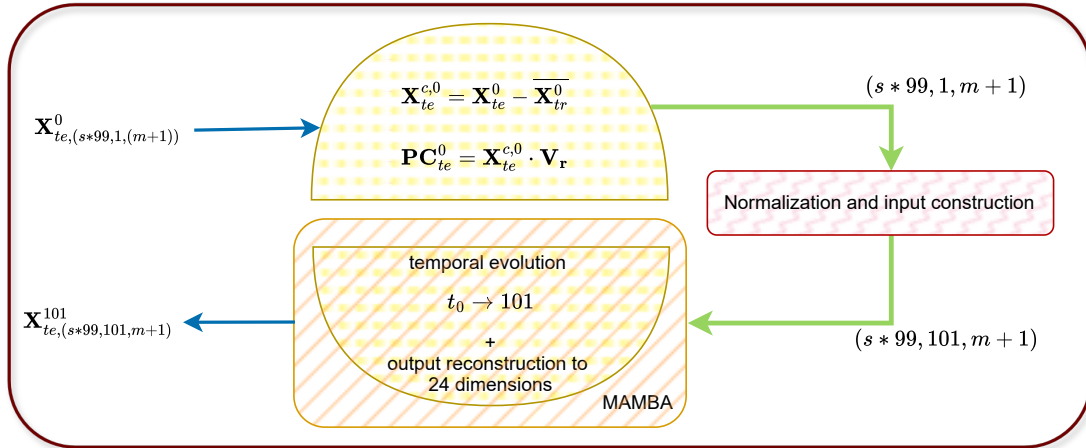


Fig. 2: **Schematic representation of latent Kinetic-Mamba:** Corresponding to the GRI dataset, we input the initial conditions corresponding to 24 state variables (temperature, mass fractions of 23 active species) into our latent-KM model, and obtain their temporal evolution up to the 100th time step. Recall that we have already performed time decomposition on our dataset before we send it to the latent-KM model. Inside the latent-KM model, our state variables are transformed into latent variables, following the PCA procedure detailed above. They are then evolved by n_t time steps and reconstructed by the Mamba block. This reconstruction happens due to the fact that our KM model is trained with output variables lying in the normalized time-decomposed 24-dimensional manifold.

2.3 Training Methodology and Error Metrics

We train our Kinetic-Mamba model using the time decomposition technique, as discussed in Section 2.1 with tiled initial conditions as input. The input to the Kinetic-Mamba is given on the time grid over which the prediction is expected. We train all our Mamba models within the Kinetic-Mamba framework using the mean squared error or MSE loss function with Adam [32] optimizer and LambdaLR learning rate scheduler. The initial learning rate is set to 10^{-3} or 10^{-4} ,

depending on the problem under consideration. We also assume the batch size to be 256, unless stated otherwise.

At inference, we obtain the predictions from the trained model given the initial conditions from the test dataset. We reconstruct the predictions across the time domain consisting of 99 segments of 101 time steps. The reconstruction is performed by concatenating the segments along the time dimension including common time points between any two segments. To evaluate the accuracy of the prediction, we then use the following metrics to compute the error between the prediction and the true values of the state variables.

Suppose that $y, \hat{y} \in \mathbb{R}_{\geq 0}^{(M \times (n_t - 1) \times (m + 2))}$ such that y corresponds to the true values of the test dataset, while $\hat{y}$ corresponds to the predicted value, both reconstructed from the time decomposition scheme. There are three different error metrics we consider to compute the error in this paper:

$$(14) \quad \% \text{ relative } L_2 \text{ error}_{(M, m+2)} = \left(\frac{\|\hat{y} - y\|_{2,t}}{\|y\|_{2,t}} \times 100 \right)$$

$$(15) \quad \overline{\% \text{ relative } L_2 \text{ error}}_{m+2} = \frac{1}{M} \sum_{j=1}^M \left(\frac{\|\hat{y} - y\|_{2,t}}{\|y\|_{2,t}} \times 100 \right)$$

$$(16) \quad \overline{\% \text{ relative } L_2 \text{ error}} = \frac{1}{M \times (m + 2)} \sum_{i=1}^{m+2} \sum_{j=1}^M \left(\frac{\|\hat{y} - y\|_{2,t}}{\|y\|_{2,t}} \times 100 \right)$$

here $\|\cdot\|_{2,t}$ denotes the L_2 norm along the temporal dimension consisting of $99 * 101$ time points. Moreover, the subscript a,b on LHS denotes the size of the resulting matrix.

To calculate our error and plot the prediction within each segment for all segments in time decomposition setup, we plot each segment as a function of the global time within that segment and compute the relative L_2 error by flattening the time across all these segments to obtain $99 * 101$ time points for each state variable. In the Syngas problems, we omit plotting the dynamics corresponding to N_2 as it stays constant over time. We show our relative L_2 error values for all samples of each state variable over either a single or multiple independent runs in violin plots, with bulk of the violin indicating that large number of samples have an error of around that value and tapering ends representing outlier samples with their corresponding error value on y-axis.

2.4 Extrapolation

It is challenging for models to generalize beyond the state variable range that they have been trained upon, specifically if there is stiffness within the solution profile. In this study, we shall also focus on assessing the accuracy of our framework in predicting the dynamics of the state variables generated with different equivalence ratio, temperature and pressure values outside the values they were trained upon. Since it is easier for models to generalize well on out-of-distribution samples generated using equivalence ratios and temperatures sampled from a uniform distribution as compared to a random distribution, we shall assess the performance of Kinetic-Mamba framework on out-of-distribution samples generated using equivalence ratios and temperatures sampled from a random distribution. Therefore, we shall generate different extrapolation data where each dataset corresponds to different equivalence ratio or temperature range, and pressure values or their combination. We shall then employ a pre-trained Kinetic-Mamba model to predict the dynamics of the state variables using initial conditions from the aforementioned extrapolation datasets. For a comprehensive assessment of the framework capabilities, our extrapolation dataset is such that it may show different ignition regimes. Therefore, utilizing Eq. (5) on the training dataset, we partition the dataset at the same critical temperature, perform normalization, construct the input as per Section 2.1, and employ ignition regime-informed KM model consisting of the respective pre-trained models to predict the dynamics of the state variables. If the data lies completely below or above the critical ignition temperature, we only employ a single corresponding standalone KM model from the setup.

2.5 Recursive Predictions

Chemical kinetics often dominates the computational cost of reacting-flow CFD because stiff source terms must be evaluated at every cell and CFD timestep. To advance the thermochemical state over multiple CFD time steps, recursive prediction is employed, where the output of a pre-trained ML model is repeatedly fed back as input to the model. This approach integrates naturally with both fixed and adaptive CFD time-stepping. For fixed time steps, the CFD timestep can be viewed as a sliding window over the time domain, with each window encompassing a segment of fixed time points, e.g. 101 time points or 10 microseconds. For adaptive time steps, the recursive framework remains effective, as the model can be applied repeatedly over variable-length intervals, allowing the surrogate to track the kinetic trajectory accurately even when the CFD timestep changes dynamically. Using fixed timestep of 10 microseconds as an example, there are 99 segments over which we make predictions in the following fashion:

Suppose the entire trajectory that needs to be predicted consists of n_t time points from $t_0 = 0\Delta t$ seconds to $(n_t - 1)\Delta t$ seconds. We start at t_0 , construct our grid of 101 time points by tiling the initial conditions as mentioned in Section 2.1 and then input it to the pre-trained model which predicts the values at those temporal grid points. To advance to the next window, we map the predicted value at the last time point of the last segment from the output normalization space to the branch input normalization space within the power-transformed space, via an inverse-forward transform, to account for different min-max transformations of branch input and model output space. We tile the result across the next 101 time points and feed it into the pre-trained model. The process continues until we reach the last window containing the desired last time point.

An analogous process is applied when using latent KM model to predict recursively, with an additional step at each window boundary point. Herein, we take predictions of the last or rather the 101-st time point of each window, which corresponds to $\mathbf{X}_{tr}^{101}$, apply an inverse-forward transform and obtain $\mathbf{PC}_{tr}^{101}$ by performing dimensionality reduction as outlined in Section 2.2.1. The resulting latent vector is tiled and fed to the pre-trained latent KM model for the next window. We repeat this process until we have predicted on the $(m + 1)$ -dimensional manifold up to all 9,901 time points. However, when the time steps are not fixed, that is, when we have variable length temporal windows, we tile the initial conditions or the last point of the previous window up to the length of the desired next window. This adaptivity with Kinetic-Mamba model makes it robust for such predictions even though it was trained on segments or windows of fixed length 101. Once we have obtained our predictions for each window, we concatenate them across the temporal dimension, removing repeated time points (corresponding to first point of prediction for all but first window) to obtain a complete trajectory on the given time domain.

3 Computational Examples

We demonstrate the accuracy of our Kinetic-Mamba framework on two problems: the Syngas (synthetic gas) mixture and the GRI-Mech 3.0 [33] scheme. The dataset we obtained for Syngas is generated from two different distributions and hence, corresponding to this mechanism, we demonstrate the accuracy of our model on the two datasets for this mechanism by employing standalone Mamba, ignition regime-informed Mamba, and the mass-conserving Mamba model on it. To show the efficiency of latent Kinetic-Mamba model in accurately predicting the dynamics as well as reconstructing the dataset to the original $(m + 1)$ -dimensional manifold, we employ it to the GRI-Mech 3.0 dataset due to a higher number of species involved in the chemical reaction. To further test the capabilities of Kinetic-Mamba framework, we extrapolate on out-of-distribution samples for Syngas B dataset. We also perform recursive prediction on Syngas A as well as GRI-Mech 3.0 datasets.

3.1 Time Decomposition

A Syngas problem was selected to develop, test, and demonstrate the workflow for building Kinetic-Mamba models for a stiff chemical kinetic mechanism. The case involves a skeletal

Syngas mechanism for CO/H₂ combustion, consisting of 11 species and 21 reactions. This mechanism represents a key subset of larger hydrocarbon fuels. The dataset consists of the mass fractions of 11 species, temperature and pressure or density. The species under consideration are H₂, O₂, O, OH, H₂O, H, HO₂, CO, CO₂, HCO, N₂. We consider two datasets corresponding to this mechanism for analysis. In one configuration, pressure remains constant during the combustion process, while temperature and density undergo temporal evolution. Thus, for this case, the data was derived from a uniform distribution of temperature and equivalence ratios. In another configuration, the density remains constant while pressure and temperature undergo evolution with time. In this study, the data were derived from a random distribution of temperature and equivalence ratios within a constant volume reactor. In instances where temperature-pressure or temperature-density predictions are made along with the mass fraction of 11 species, the equation of state (EOS) can be employed to recover the density or pressure state variable. Therefore, the Syngas problem consists of 13 state variables. Henceforth, we shall refer to the aforementioned datasets as “Syngas A” and “Syngas B”, depending on whether they are from the constant pressure or constant volume reactor, respectively. The minimum and maximum values corresponding to the training and testing datasets are tabulated in Section A.

3.1.1 Standalone Mamba Model

For a Syngas A model, we consider a syngas fuel composed of CO/H₂ with mole fractions CO = 0.5 and H₂ = 0.1, burned in air represented by an oxidizer mixture O₂ = 0.233 and N₂ = 0.767. The training data is generated using the Cantera library with an initial temperature in the range of [1000 K, 1500 K] and an equivalence ratio range of [0.7, 1.3]. The dataset consists of mass fractions of 11 species, temperature and density that vary across all time steps for all samples. The dataset also consists of pressure, which stays constant over time but has different values for different samples. For our model, we consider 13 thermochemical state variables comprising density, temperature, mass fractions of 11 species. Pressure at any instant for any sample can be recovered using the equation of state. Our training dataset consists of 6000 samples across 10002 time points for 13 state variables, and testing dataset consists of 1500 samples across 10002 time points for the same state variables. We then scale this dataset onto $[-1, 1]^{13}$ and perform time decomposition by segmenting the time domain into 99 segments of temporal length 101. The model is trained with 321,693 parameters on the training dataset for 6001 iterations. For training the model, we use the Adam [32] optimizer with LambdaLR learning rate scheduler and a learning rate of 10^{-3} . We use MSE loss function and a batch size of 256. Fig. 3 shows the performance of the kinetic-mamba model on arbitrary test data sample. Across three runs, we reconstructed the output to the physical space over $99 * 101$ time points and computed the percentage relative L_2 error across all time steps according to Eq. (14). Corresponding to these runs, Fig. 4 shows these error values. The mean of this percentage relative L_2 error yielded the error corresponding to one run as given in Eq. (16). Over three runs, we obtained an average error of 0.015%. The mean and standard deviation for all species is obtained by computing the mean of the percentage relative L_2 error across all samples (as per Eq. (15)) and then calculating its mean and standard deviation across three runs. We tabulate this information in Table 1.

3.1.2 Mass-conserving Kinetic-Mamba Model

The sum of the mass fractions of all chemical species adds up to 1 to ensure that the mass has been conserved over the course of the entire reaction. To do so, we enforce the conservation of mass constraint on 11 species involved in the reaction by projecting them onto a 10-dimensional manifold, bringing down the number of variables to 12. This has been discussed in Section 2.2. We then feed the initial conditions of the 12 variables tiled over the time domain of 101 time points, and train it with a standalone Mamba model using 321,532 parameters. We use learning rate of 10^{-3} and LambdaLR learning rate scheduler with Adam [32] optimizer for 6,001 iterations. Upon reconstruction of the time domain, we plot the relative L_2 error across the time domain given by Eq. (14) in Fig. 5. We observed a mean relative L_2 error given by Eq. (16) across three runs as 0.017%. We tabulate the mean and standard deviation of the error values obtained using Eq. (15) for three runs in Table 2. We plot an arbitrary sample showing the predicted versus true values of the dynamics of state variables segment-wise in Fig. 6.

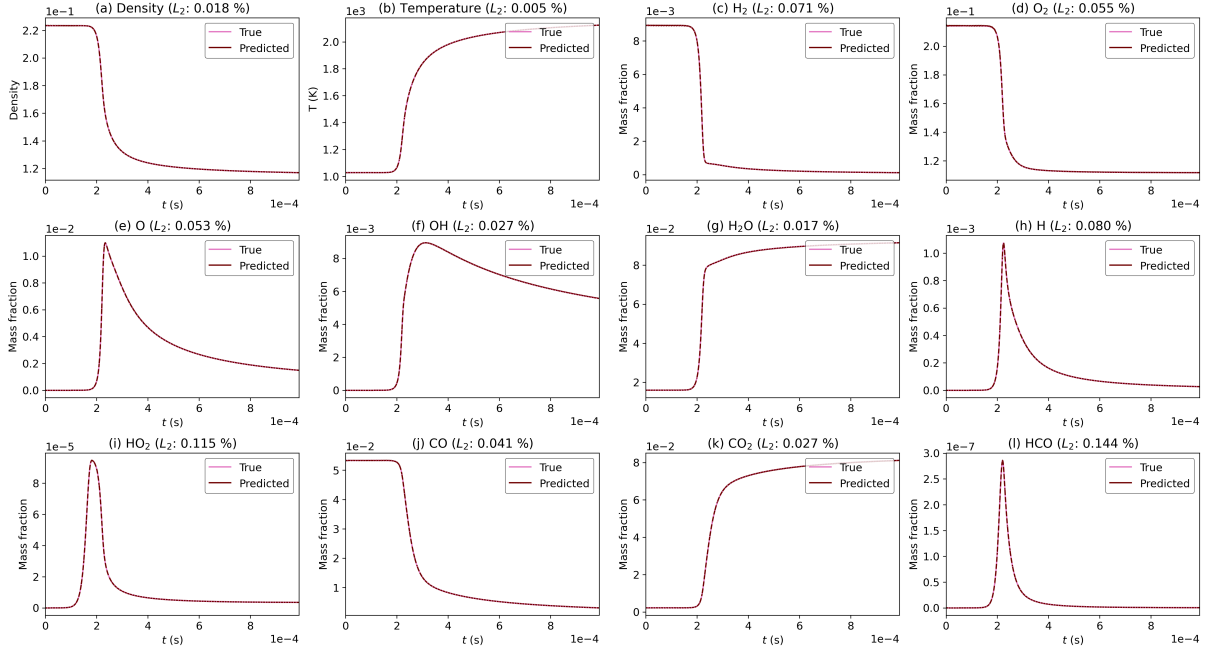


Fig. 3: Sample from Syngas A mechanism: We can see good agreement between the predicted dynamics of the state variables and the ground truth values of the dynamics for an arbitrary test sample.

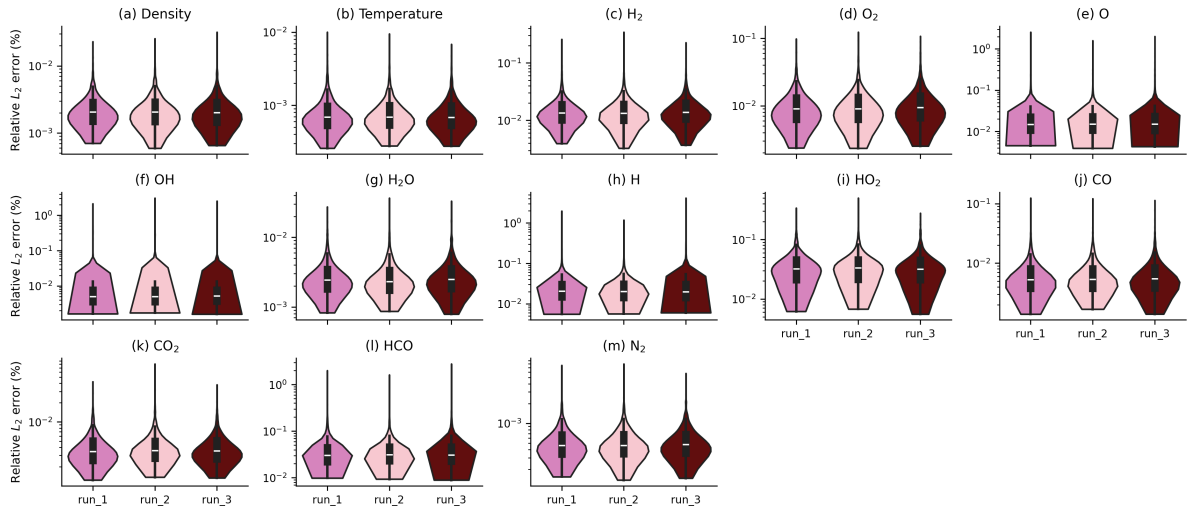


Fig. 4: Violin plot for the Syngas A mechanism: This figure represents the percentage relative L_2 error obtained when the L_2 norm is taken with respect to the temporal dimension for both predicted and true test dataset. The plot in the log scale represents good accuracy across all samples for all 13 state variables.

Table 1: **Syngas A Problem:** Mean and standard deviation of percentage relative L_2 error values for Syngas A Mamba model across three independent runs.

State Variables	μ	σ
Density	2.468e-3	2.040e-5
Temperature	8.703e-4	3.159e-6
H ₂	1.685e-2	3.423e-4
O ₂	1.158e-2	2.754e-4
O	2.558e-2	4.887e-4
OH	1.514e-2	2.417e-4
H ₂ O	3.048e-3	6.947e-5
H	3.121e-2	1.357e-3
HO ₂	3.720e-2	3.153e-4
CO	7.091e-3	6.028e-5
CO ₂	4.423e-3	4.026e-5
HCO	4.388e-2	1.041e-3
N ₂	5.826e-4	1.130e-6

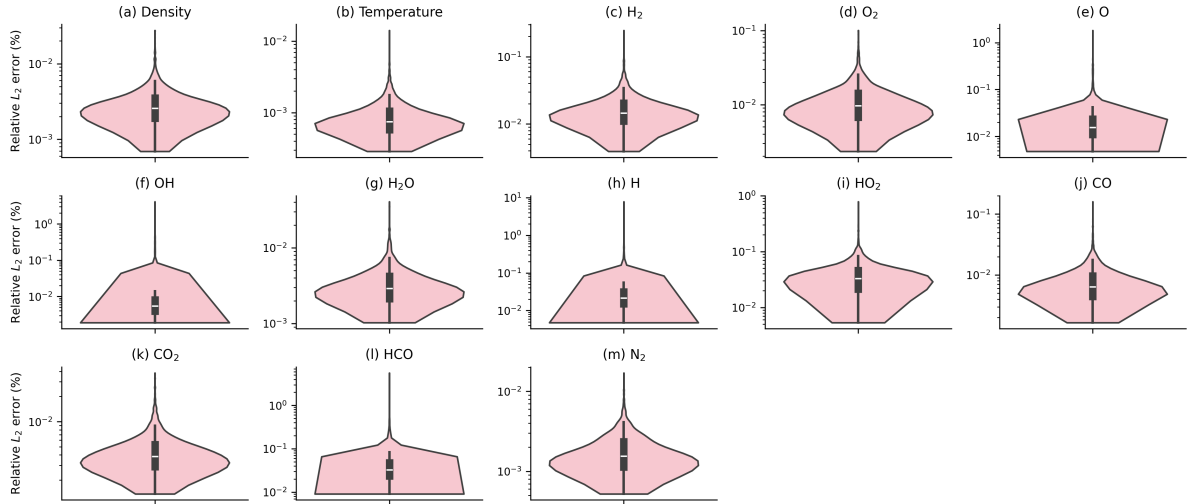


Fig. 5: **Violin plot for the Syngas A mechanism using mass-conserving KM model:** This figure represents the percentage relative L_2 error obtained when the L_2 norm is taken with respect to the temporal dimension for both predicted and true test dataset. The plot represents good accuracy across all samples for all 13 state variables, as seen above in the log scale. The total relative L_2 error is 0.017%.

Table 2: **Syngas A (Mass-Conserving) Problem:** This table shows the mean and standard deviation of the percentage relative L_2 error values over three runs for Mass-conserving Mamba model over the Syngas A dataset.

State Variables	μ	σ
Density	2.990e-3	8.715e-5
Temperature	9.335e-4	3.686e-6
H ₂	1.820e-2	1.148e-4
O ₂	1.214e-2	1.629e-4
O	2.624e-2	4.519e-4
OH	1.592e-2	1.006e-3
H ₂ O	3.731e-3	9.375e-5
H	3.433e-2	1.186e-3
HO ₂	3.905e-2	3.778e-4
CO	8.235e-3	1.118e-4
CO ₂	4.876e-3	1.077e-4
HCO	4.715e-2	9.453e-4
N ₂	1.945e-3	2.208e-5

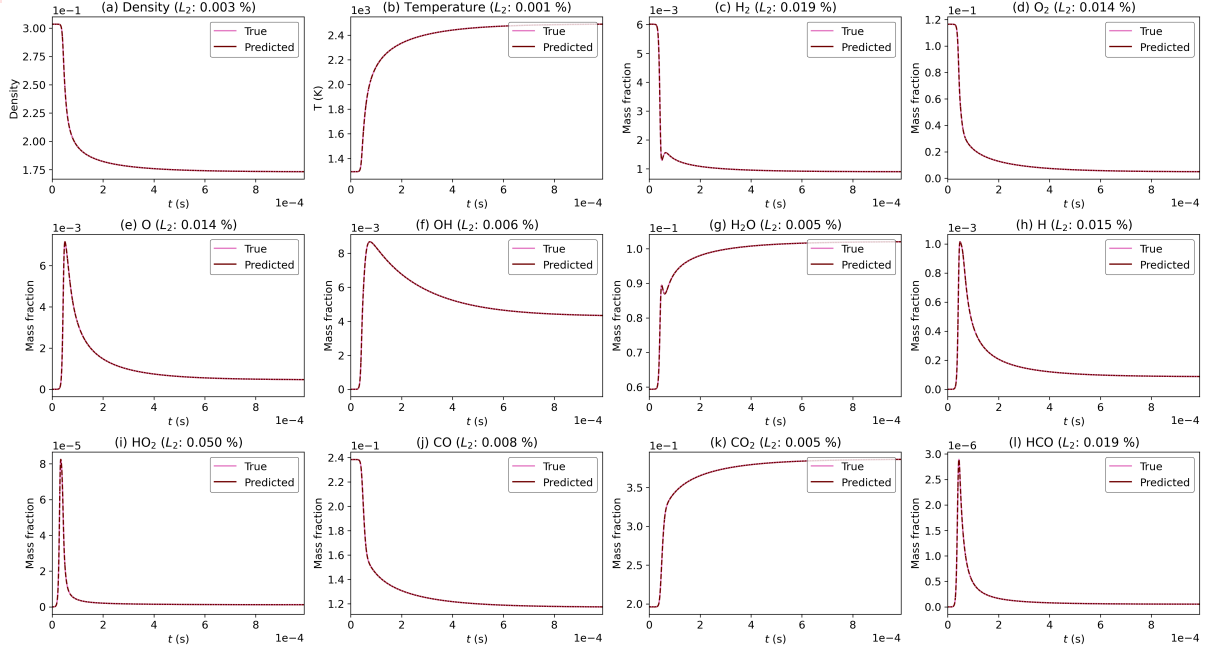


Fig. 6: **Sample from Mass-conserving Kinetic-Mamba framework for Syngas A mechanism:** Plot of an arbitrary sample showing good agreement with ground truth values from the syngas A dataset when obtained using mass-conserving Kinetic-Mamba standalone model.

3.1.3 Ignition Regime-informed Kinetic-Mamba Model

Our Syngas B fuel mixture contains 50% CO, 10% H₂, and 40% N₂ by volume, while the oxidizer contains 25% O₂ and 75% N₂ by volume. This dataset is generated using a zero-dimensional constant-volume Cantera reactor. Initial equivalence ratios were randomly sampled from $\phi_0 \in [0.7, 1.4]$, and initial temperatures were randomly sampled from $T_0 \in [500, 2840]$ K. A total of 10,000 samples were generated, each containing 10,000 uniformly spaced time-points ($\Delta t = 10^{-7}$) s and the value at the first time point here is same as Δt . Additionally, it has temperature profiles that exhibit igniting vs non-igniting regimes during their course of evolution within the reaction.

During the evolution of the mass fractions with time, temperature may often represent varied dynamics, such as certain profiles below a threshold remaining constant over time, while others increase and then reach a steady state. In short, above a certain threshold, called the critical ignition temperature, the temperature profiles may rise before plateauing, while below the threshold, the profiles remain flat. We refer to the two regimes as ignition and non-ignition regimes, respectively. We identify this threshold τ K using the formulation outlined in Eq. (5) with denominator equal to 9,999. We split the dataset at τ into two parts, reflecting the two different ignition regimes. Corresponding to the initial temperature values below τ K we have 1,400 samples in the training dataset and 350 samples in the testing dataset, while for the initial temperature values above τ K, we have 6,600 training samples and 1,650 test data samples. In our case, the value of τ K is roughly 889.09 K.

We train two standalone Mamba models with 231,757 and 384,943 parameters. Each model was trained using MSE loss criterion with learning rate 10^{-4} , Adam [32] optimizer and LambdaLR learning rate scheduler. The first model corresponding to constant temperature profiles was trained for 2,501 epochs with batch size of 256, while the second model was trained for 5,001 epochs with a batch size of 256×2 . The outputs from both the models were reconstructed to 99×101 point time domain and then concatenated along the sample direction. We then computed the relative L_2 error across all time steps as given in Eq. (14). We show the true vs predicted values in the physical domain for temperature values below τ K in Fig. 7 and above τ K in Fig. 8. The mean of the percentage relative L_2 error was taken, as per Eq. (16), and average was computed across three runs. We obtained a total error of 0.023%. The mean and standard deviation of the error values for each state variable according to Eq. (15) is also tabulated in Table 3 for three independent runs. Fig. 9 shows the relative L_2 error for all samples of each species when predictions were obtained from the Ignition Regime-informed Kinetic-Mamba model. We also observed that if this division in temperature profile was not performed to employ different standalone Mamba models, then the average error over three independent runs was 0.282% when training was performed for 5,001 iterations with a batch size of 256 and learning rate of 10^{-4} using the same optimizer and learning rate scheduler with 384,943 parameters.

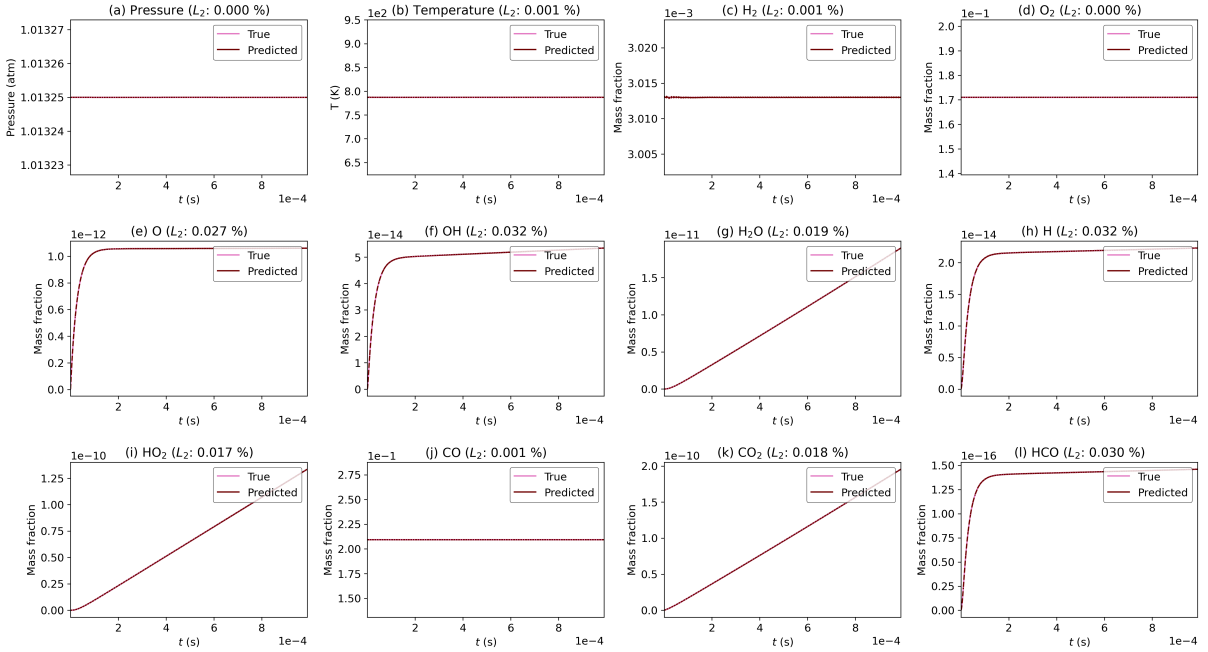


Fig. 7: Sample 1 from Syngas B mechanism: We can see good agreement between the predicted dynamics of the state variables and the ground truth values of the dynamics for an arbitrary test sample lying in the non-ignition regime. We have expanded the y-axis in this figure to improve the visualization by removing small fluctuations due to constant values.

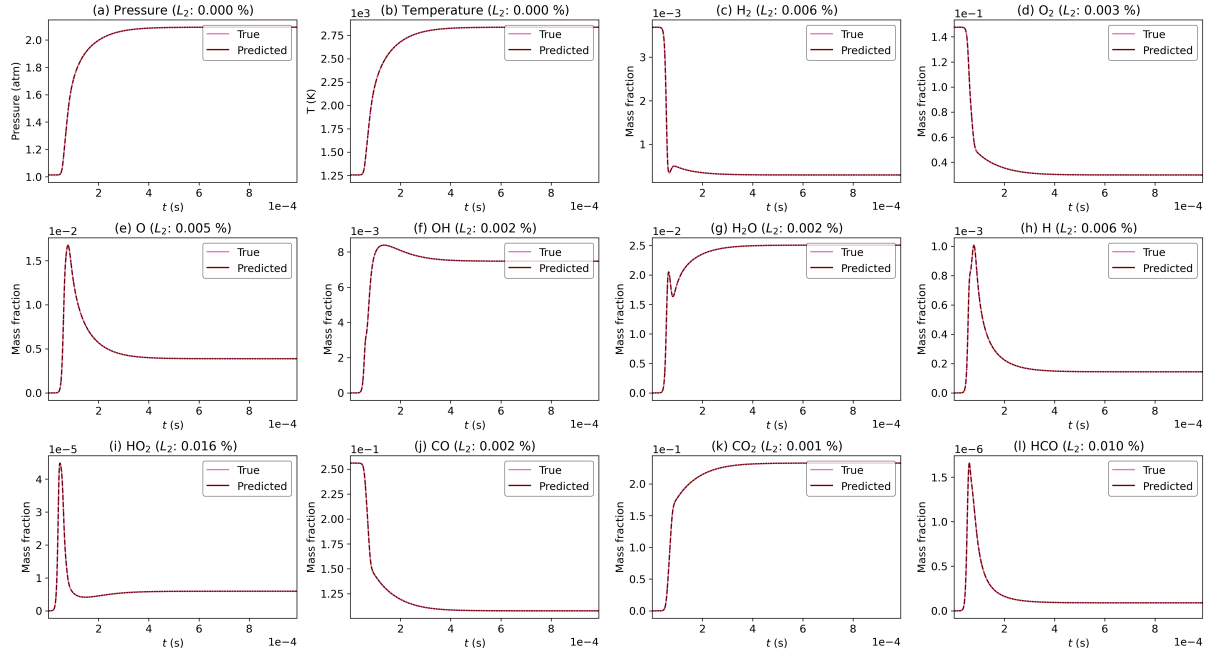


Fig. 8: **Sample 2 from Syngas B mechanism:** We can see good agreement between the predicted dynamics of the state variables and the ground truth values of the dynamics for an arbitrary test sample lying within the ignition regime.

Table 3: **Syngas B Problem:** Mean and standard deviation of the percentage error values over three runs for Syngas B Mamba model.

State Variables	μ	σ
Pressure	3.682e-4	4.816e-6
Temperature	5.952e-4	1.403e-5
H ₂	3.946e-3	2.113e-5
O ₂	1.842e-3	1.872e-5
O	4.709e-2	2.583e-3
OH	5.159e-2	3.900e-3
H ₂ O	3.108e-2	4.360e-4
H	5.490e-2	4.066e-3
HO ₂	2.977e-2	1.681e-3
CO	1.266e-3	4.753e-5
CO ₂	2.490e-2	4.379e-4
HCO	5.861e-2	5.189e-3
N ₂	5.814e-5	1.585e-7

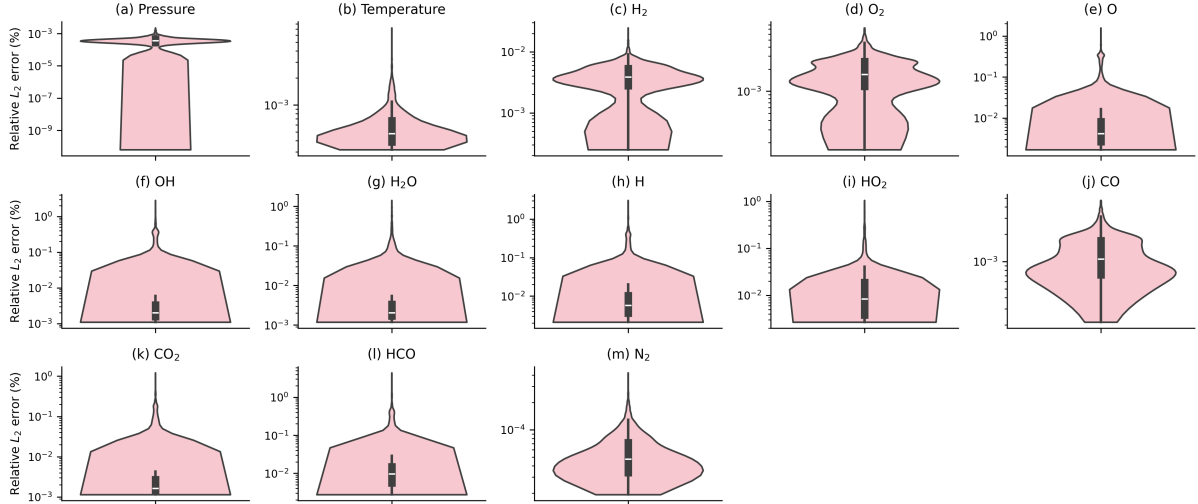


Fig. 9: **Violin plot for the Syngas B mechanism:** This figure represents the percentage relative L_2 error obtained when the L_2 norm is taken with respect to the temporal dimension for both predicted and true test dataset. The plot is in log-scale on the y-axis. The plot refers to the Ignition Regime-informed Kinetic-Mamba standalone model which takes the ignition regime in the dataset into consideration.

3.1.4 Latent Kinetic-Mamba Model

The GRI-Mech 3.0 [33] is an optimized mechanism for methane/air combustion. It consists of 53 species and 325 reactions. For the sake of our models, we only consider 23 species which have strictly positive mass fractions. Thus, we have 24 state variables including temperature and the 23 species' mass fractions. We partitioned the dataset into 2,080 samples of training dataset and 520 samples of testing dataset. In this problem as well, we divide our time series into 99 segments of length 101, that is, we perform time decomposition and then normalized the dataset as per Section 2.1. We then converted our state variables into latent variables following Section 2.2.1. While we trained our model and predicted on latent manifold of latent dimension 12, we used output labels from the original 24-dimensional manifold, which were also normalized and time decomposed as per Section 2.1. We trained the model with 232,408 parameters, batch size of 256 for 6,001 iterations using Adam [32] optimizer and LambdaLR learning rate scheduler with learning rate 10^{-3} . We computed the mean of the percentage relative L_2 error over the whole time domain of 101×99 time points as per Eq. (16) for three runs as 0.010%. We tabulate the mean and standard deviation of the errors computed using Eq. (15) for three runs in Table 4 and plot the percentage error values across all time steps for all samples of each state variable in Fig. 10. We also plot the true values of the dynamics of state variables versus predicted values of the dynamics of the state variables obtained from our latent Kinetic-Mamba model in Fig. 11.

3.2 Extrapolation

We discussed our method for performing extrapolation in Section 2.4. In this section, we shall further outline the method as well as provide results of performing extrapolation using ignition regime-informed KM model. We use our pre-trained model as defined in Section 2.2 and then input the extrapolation datasets one by one, assuming same critical temperature as determined during the training process to assess the extrapolation accuracy. For this study, we generated four extrapolation datasets corresponding to the following initial temperature T_0 , equivalence ratio ϕ and pressure P values:

1. 20 samples with T_0 within $[2840, 3040]K$, $\phi \in [0.7, 1.4]$ and pressure of 1 atm.
2. 30 samples with $T_0 \in [500, 2840]K$, $\phi \in [0.5, 0.7]$ and pressure of 1 atm.

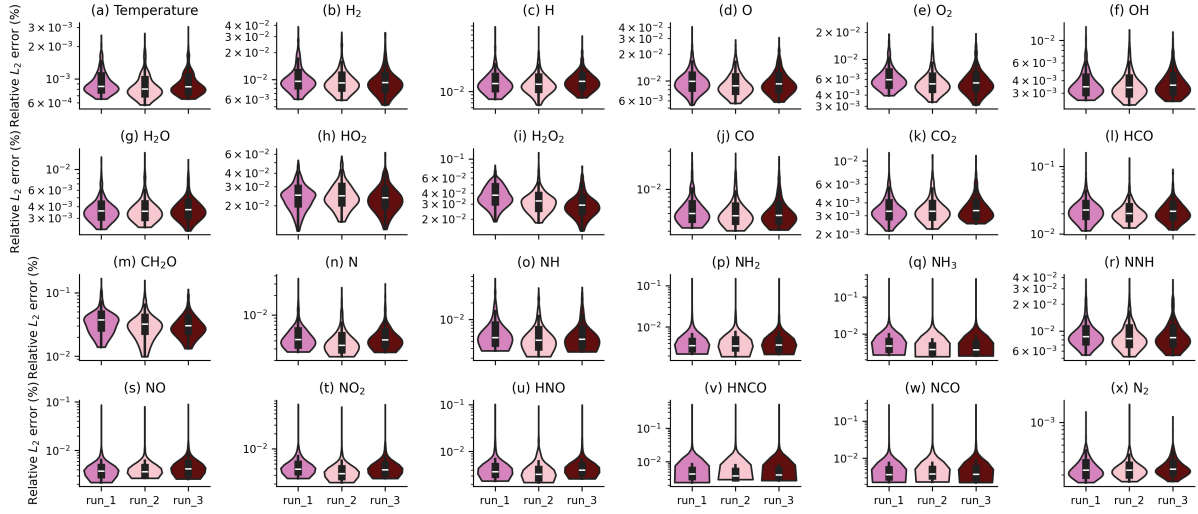


Fig. 10: Violin plot for the GRI mechanism: This figure represents the percentage relative L_2 error obtained when the L_2 norm is taken with respect to the temporal dimension for both predicted and true test dataset. The plot represents good accuracy across all samples for all 24 state variables. (The plot has error values depicted in the log-scale on y-axis.)

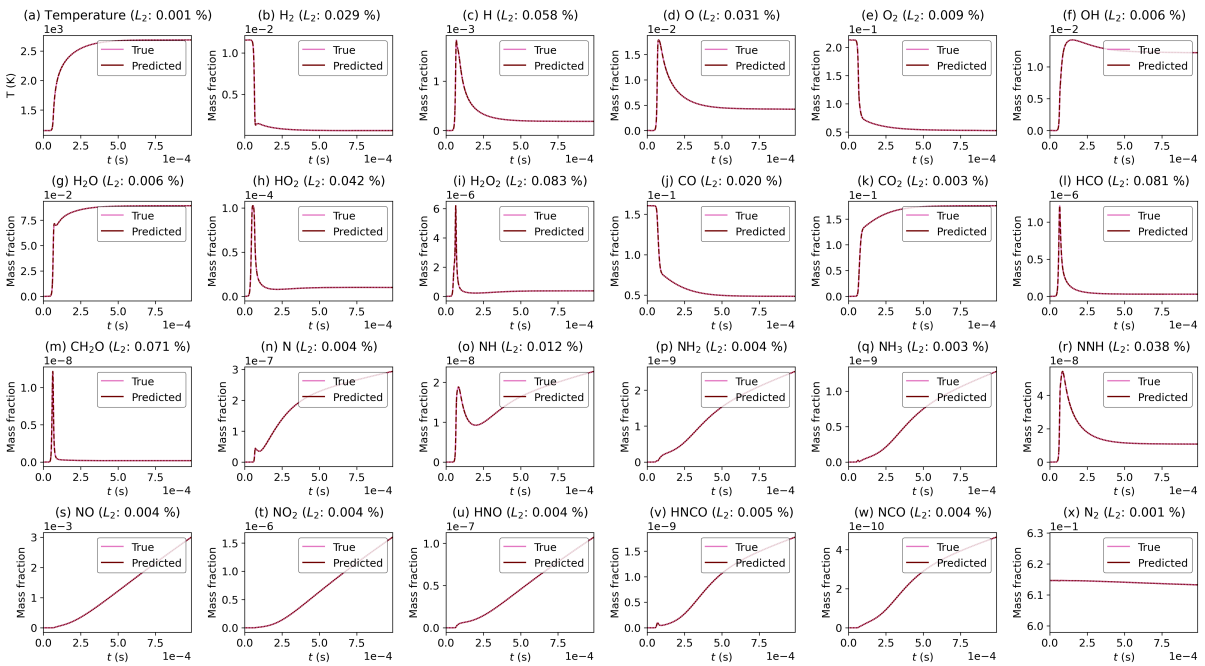


Fig. 11: Sample from GRI-Mech 3.0 mechanism: We can see good agreement between the predicted dynamics of the state variables and the ground truth values of the dynamics for an arbitrary test sample.

Table 4: **GRI-Mech 3.0 Problem:** Mean and standard deviation of the percentage error values over three runs for latent Kinetic-Mamba model.

State Variables	μ	σ
Temperature	9.383e-4	2.768e-5
H ₂	1.062e-2	3.687e-4
H	1.462e-2	4.583e-4
O	1.033e-2	3.490e-4
O ₂	6.275e-3	2.855e-4
OH	3.935e-3	4.240e-5
H ₂ O	3.969e-3	4.677e-5
HO ₂	2.563e-2	5.759e-4
H ₂ O ₂	3.529e-2	3.523e-3
CO	5.911e-3	1.720e-4
CO ₂	3.667e-3	3.392e-5
HCO	2.403e-2	1.095e-3
CH ₂ O	3.707e-2	2.549e-3
N	4.926e-3	2.343e-4
NH	6.796e-3	5.717e-4
NH ₂	5.207e-3	9.980e-5
NH ₃	6.310e-3	4.782e-4
NNH	9.781e-3	1.869e-4
NO	4.566e-3	2.943e-4
NO ₂	4.396e-3	3.848e-4
HNO	4.859e-3	2.041e-4
HNCO	6.851e-3	1.475e-4
NCO	5.590e-3	1.915e-4
N ₂	2.586e-4	5.212e-6

3. 32 samples with $T_0 \in [500, 2840]K$, $\phi \in [1.4, 2.0]$ and 1 atm pressure.

4. 34 samples with $T_0 \in [500, 2840]K$, $\phi \in [0.7, 1.4]$ and pressure $\in [1, 1.5]$ atm.

We plot arbitrary samples from these datasets in Fig. 12, Fig. 13, Fig. 14, Fig. 15, respectively. The violin plot Fig. 16 shows the accuracy obtained when we extrapolated using pre-trained Kinetic-Mamba models discussed in Section 3.1.2 for all samples of all the extrapolation datasets. We note that the model performs well for out-of-distribution temperature values but has some deterioration as the range equivalence ratio is changed. The performance worsens further with increasing pressure value but remains reasonable for most samples. Many species have very small values and sometimes the error value though small is overshadowed by an even smaller denominator d (true value) during computation of relative L_2 error as per Eq. (14) causing dramatic inflation in error values. We therefore, clip the denominator by $\max(\epsilon, d)$ and revise the error metric by clipping the denominator to get reasonable values as detailed in Section C.

3.3 Recursive Predictions

We highlighted our methodology in Section 2.5 to carry out recursive predictions. In the following subsections, we will discuss our results of recursive prediction using the pre-trained Kinetic-Mamba models discussed above. The recursive prediction setting is considerably more

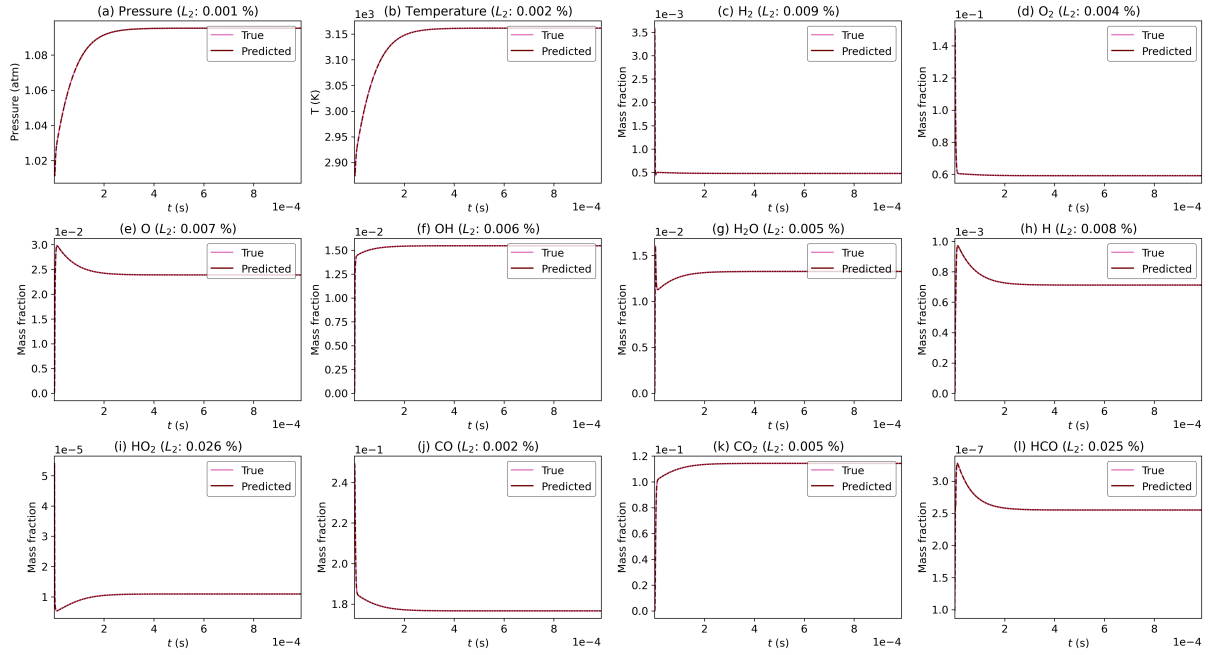


Fig. 12: **Extrapolation Sample 1 for Syngas B:** Plot of predicted vs true data of an arbitrary sample from extrapolation dataset Item 1.

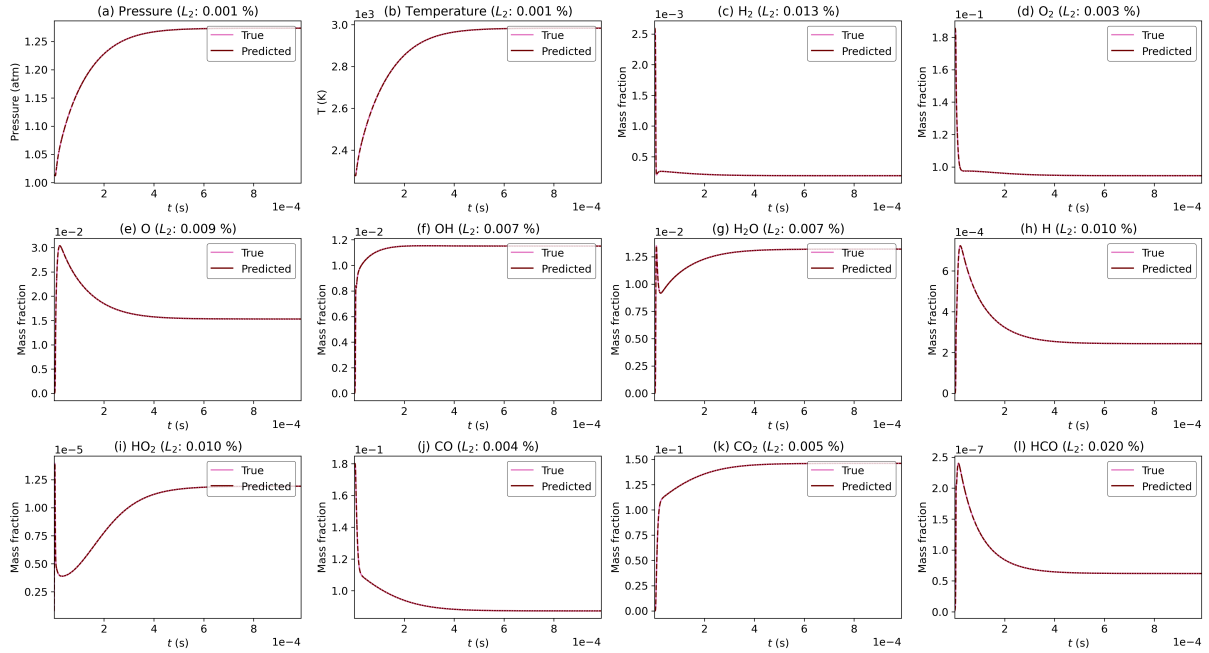


Fig. 13: **Extrapolation Sample 2 for Syngas B:** Plot of predicted vs true data of an arbitrary sample from extrapolation dataset Item 2.

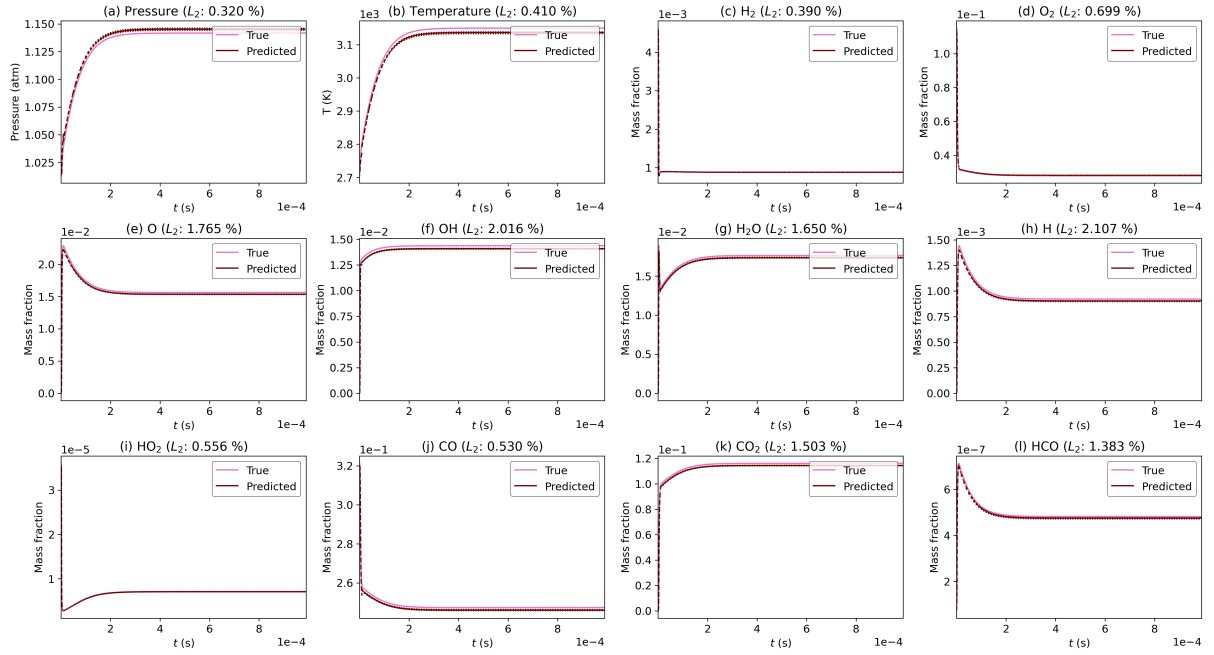


Fig. 14: **Extrapolation Sample 3 for Syngas B:** Plot of predicted vs true data of an arbitrary sample from extrapolation dataset Item 3.

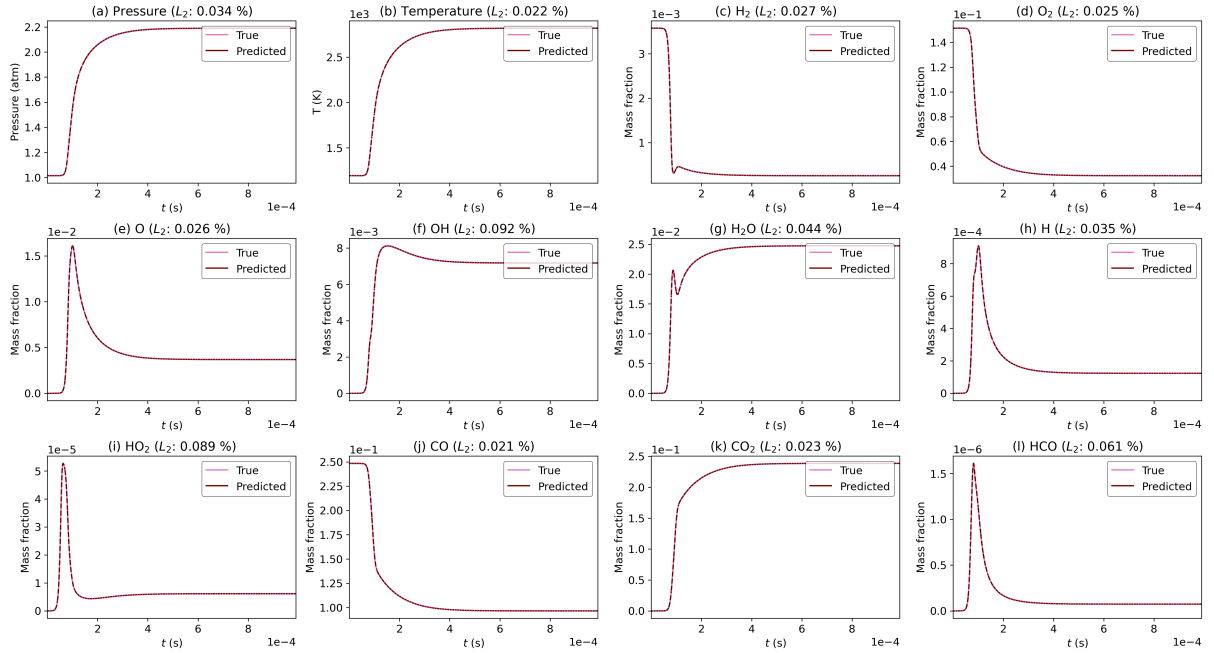


Fig. 15: **Extrapolation Sample 4 for Syngas B:** Plot of predicted vs true data of an arbitrary sample from extrapolation dataset Item 4.

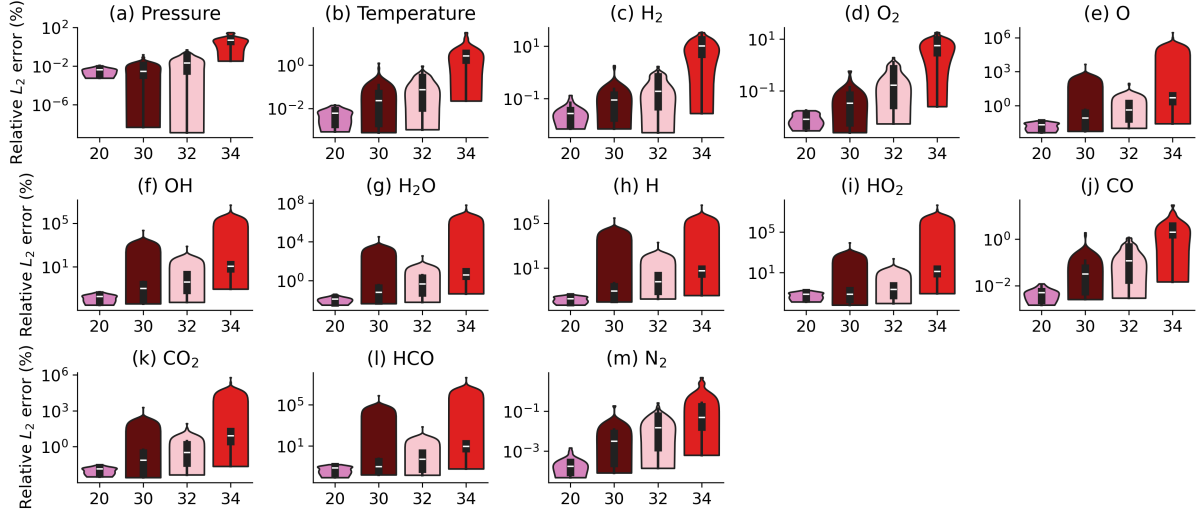


Fig. 16: **Extrapolation for Syngas B KM model:** Violin plot obtained when pre-trained ignition regime-informed KM model is employed on extrapolation datasets Item 1, Item 2, Item 3, Item 4 containing 20,30,32, and 34 samples respectively. Here, the error values are plotted in log-scale.

challenging than one-step prediction, as errors can accumulate, amplify, as well as propagate across successive steps.

3.3.1 Syngas A Problem with Kinetic-Mamba Framework

For the recursive predictions, we use our pre-trained standalone Mamba model as described above with same hyper-parameters and parameters. We then load this model to perform recursive predictions on our test dataset. For this, we proceed to predict recursively in windows of length 101, i.e., we start with initial conditions at t_0 and predict until t_{100} . We use the predicted state at t_{100} as input to the next window via the inverse-forward transform described in Section 2.5, repeating for all 99 windows. The concatenated trajectory spans 9,901 unique time points after removing boundary duplicates. We also extract the first 9,901 time points from the test dataset to compute the error. We decode our output back to the physical space. For every sample of each state variable within the test dataset, we tabulate the error values in Table 5 to show the relative L_2 error across three different runs. Over these three independent runs, we obtain a relative L_2 error of $(1.898 \pm 0.481)\%$. We plot the true test data corresponding to an arbitrary sample and its recursive prediction in Fig. 17.

3.3.2 GRI-Mech 3.0 Problem with Latent Kinetic-Mamba Framework

A key advantage of the Kinetic-Mamba model is its ability to perform adaptive temporal evolution during prediction. Rather than requiring a fixed window of 101 time points at each recursive step, the model is able to yield predictions over segments of variable temporal lengths as detailed in Section 2.5. For instance, given CFD time steps of 10, 7.5, 3.0, . . . microseconds, one can predict over 101 time points, feed the final prediction as input, then predict the next 76 time points, and continue with a segment of length 31, and so on. Fig. 18 illustrates this approach for an arbitrary sample, comparing true and predicted values across variable-length temporal segments, spanning a total of 3,405 time points. Over three independent predictions using three pre-trained Latent KM models, we obtain a mean relative error of $(4.51 \pm 1.12)\%$.

3.3.3 Directions for Improvement

The KM models efficiently predict the dynamics of the state variables in time decomposition, extrapolation and recursion. However, addressing the following can further reduce the relative error on recursive predictions.

Table 5: **Syngas A: Recursive Prediction:** Mean and standard deviation of the percentage error values over three runs for recursive predictions obtained using pre-trained Kinetic-Mamba model.

State Variables	μ	σ
Density	1.102e-1	5.100e-3
Temperature	1.169e-1	8.379e-3
H ₂	2.891e-1	1.285e-2
O ₂	2.479e-1	1.037e-2
O	6.538	2.206
OH	9.098	2.172
H ₂ O	2.047e-1	2.161e-2
H	4.350	1.591
HO ₂	1.052	6.315e-2
CO	1.621e-1	5.943e-3
CO ₂	1.280e-1	8.721e-3
HCO	2.358	4.326e-1
N ₂	1.411e-2	1.544e-4

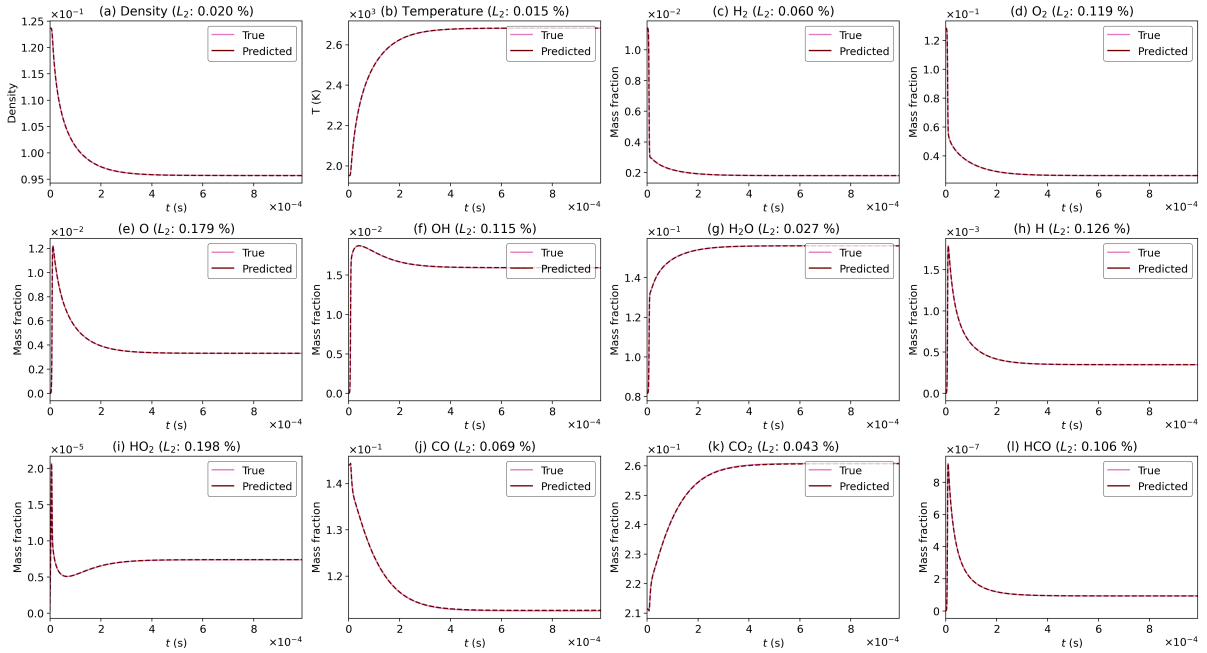


Fig. 17: **Recursive prediction using KM model on Syngas A:** We used pre-trained KM model to perform recursive predictions over 99 windows of length 101.

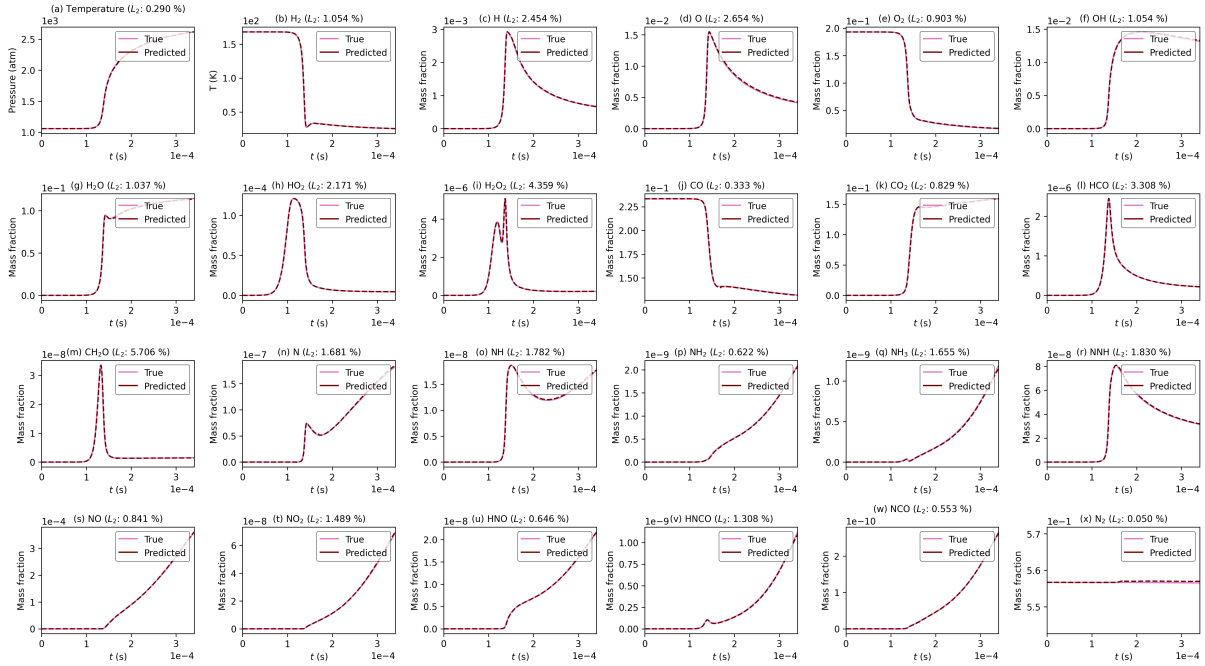


Fig. 18: Adaptive time step prediction using latent KM model on GRI-Mech 3.0: We used latent KM model to make recursive predictions at intervals of different lengths using last time step of previous interval as input to next time step.

The inverse-forward transform applied at each window boundary point introduces small numerical errors that may get amplified over subsequent roll-outs. We believe that training in double precision can help substantially reduce these error values when inference is performed recursively. Additionally, augmenting the feature dimension with explicit time values, similar to the input construction for Fourier Neural Operators (FNOs) [14], can improve accuracy. However, this also introduces a possibility that the model could suffer or fail to demonstrate any drastic improvement in the accuracy, owing to the model not being able to accurately connect the temporal values with predictions within time windows of length 101, causing it to either learn spurious patterns or overfit to the dynamics of the local window on which it was trained upon. Strategies such as randomizing the time window offsets during model training can help tackle this. Moreover, training the KM models using adaptive loss functions especially designed to account for accuracy at each roll-out, or a better training methodology for including time as an additional feature can help in improving recursive predictions obtained using a pre-trained model (trained in a time decomposition setting).

Other directions, such as using robust error metrics that consider that relative L_2 errors can be dominated by near-zero denominators; understanding the means to augment normalized global or local temporal information or adopting a different methodology when performing the roll-outs are left for future work.

4 Conclusion

In this paper, we propose a framework called Kinetic-Mamba. The Kinetic-Mamba framework is capable of accurately predicting the dynamics of stiff chemical systems using only the initial conditions as input. It also takes varied ignition regimes within the dynamics of temperature into consideration by employing different Mamba models to different regimes within the dataset. We also implemented a latent Kinetic-Mamba model, which uses principal component analysis to reduce the dimension of the state variables by converting them into latent variables, and then evolves them in time while also reconstructing the predictions back to the original state variables. We demonstrated these capabilities of the framework through Syngas and GRI-Mech 3.0 datasets where we performed training in time decomposition fashion and obtained predictions in both time-decomposed manner as well as recursively. We also observed good agreement between the

predicted and the true values in extrapolation using wide range of out-of-distribution datasets that were generated using parameters that lay outside our training range. Moreover, our latent Kinetic-Mamba model performed well when trained on 99 segments of length 101 but made to recursively predict on time segments of different lengths. Through these applications of Kinetic-Mamba model, we demonstrated the accuracy of selective state space Mamba model in accurately predicting the chemistry of stiff systems.

We note that while we obtained great accuracy, the training time required by Kinetic-Mamba is much higher. In an ideal case of integration of the framework with CFD solver, this cost is amortized by the fast prediction times, since training is mostly done offline. We list the training and prediction time in Section B for our different demonstrative examples. We also note that higher training times can be attributed to the higher number of samples that the model is trained upon as a result of time decomposition method. Additionally, since the main focus of this paper is to illustrate the application of our Mamba-based Kinetic-Mamba model when applied to stiff chemical kinetics, the training was done in single precision.

As a part of future work, we want to focus on the extension of the Kinetic-Mamba framework to support double precision data resolution during network training. We also want to focus on model training strategies that can enhance the accuracy during recursive prediction. Kinetic-Mamba’s performance on recursive predictions can be ascribed to its failure to be informed about the stiffness in the dynamics and when they might occur, as well as to the skewed information that state space model in general receives due to localization of dynamics within the prediction windows. We discussed several pathways to improve accuracy in recursive predictions, such as, augmenting the feature dimension with additional information like time values, performing training with a different methodology using adaptive loss functions, specifically designed to increase accuracy on recursive predictions, as well as by training the model in double precision. We believe that these pathways can lead to interesting future directions. Future works can also focus on training parallelization strategies that can help reduce the runtime when training subsets from a bigger dataset in the presence of varied ignition regimes.

To summarize, in this study, we presented a Kinetic-Mamba framework that helps in accurately predicting the dynamics of the thermochemical state variables, via its standalone Mamba model, ignition regime-informed Mamba models, mass-conserving Mamba model and latent Mamba model, all trained in time-decomposition fashion. The Kinetic-Mamba framework is adept at time decomposed predictions as well as in extrapolation tasks, and shows reasonable accuracy in predictions obtained recursively. Through this work, we demonstrate that Mamba-based models provide a scalable alternative to implicit solvers for stiff multiscale dynamical systems. This illustrates the ability of selective state space models emerging as a cornerstone in future studies based on combustion problems involving chemical kinetics of complex reactions.

Acknowledgment

The authors acknowledge the support from the Small Business Technology Transfer (STTR) program, USA (Grant No: GR5291245) and Defense Threat Reduction Agency (DTRA), USA. This research was conducted using computational resources available at the Center for Computation and Visualization, Brown University. The implementation of standalone Mamba models is inspired by [24] and the base code of Mamba models from [23].

Data Availability Statement

Data will be made available upon request to the corresponding author.

AI Disclosure Statement

During the preparation of this work, the authors used Claude (Anthropic) to enhance the readability of the manuscript. After using the tool, the authors reviewed and edited the content as needed, and take full responsibility for the content of the publication.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] S.B. Pope. Computationally efficient implementation of combustion chemistry using in situ adaptive tabulation. *Combustion Theory and Modelling*, 1(1):41–63, 1997. doi: 10.1080/713665229. URL <https://doi.org/10.1080/713665229>.
- [2] Robert J Kee, Michael E Coltrin, and Peter Glarborg. *Chemically reacting flow: theory and practice*. John Wiley & Sons, 2005.
- [3] Colin J Aro. CHEMSODE: a stiff ODE solver for the equations of chemical kinetics. *Computer physics communications*, 97(3):304–314, 1996.
- [4] Colin J Aro. A stiff ODE preconditioner based on newton linearization. *Applied numerical mathematics*, 21(4):335–352, 1996.
- [5] Colin J Aro, Douglas A Rotman, and Garry H Rodrigue. A high performance chemical kinetics algorithm for 3-d atmospheric models. *International Journal of High Performance Computing Applications*, 13(1):3–15, 1999.
- [6] Donald Dabdub and John H Seinfeld. Extrapolation techniques used in the solution of stiff ODEs associated with chemical kinetics of air quality models. *Atmospheric Environment*, 29(3):403–410, 1995.
- [7] Joachim Rang. Improved traditional rosenbrock–wanner methods for stiff ODEs and DAEs. *Journal of Computational and Applied Mathematics*, 286:128–144, 2015. ISSN 0377-0427. doi: <https://doi.org/10.1016/j.cam.2015.03.010>. URL <https://www.sciencedirect.com/science/article/pii/S0377042715001569>.
- [8] Alisha J Sharma, Ryan F Johnson, David A Kessler, and Adam Moses. Deep learning for scalable chemical kinetics. In *AIAA scitech 2020 forum*, page 0181, 2020.
- [9] Thomas S. Brown, Harbir Antil, Rainald Löhner, Fumiya Togashi, and Deepanshu Verma. Novel DNNs for Stiff ODEs with Applications to Chemically Reacting Flows. In Heike Jagode, Hartwig Anzt, Hatem Ltaief, and Piotr Luszczek, editors, *High Performance Computing*, pages 23–39, Cham, 2021. Springer International Publishing. ISBN 978-3-030-90539-2.
- [10] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [11] Weiqi Ji, Weilun Qiu, Zhiyu Shi, Shaowu Pan, and Sili Deng. Stiff-PINN: Physics-Informed Neural Network for Stiff Chemical Kinetics. *The Journal of Physical Chemistry A*, 125(36): 8098–8106, 2021. doi: 10.1021/acs.jpca.1c05102. URL <https://doi.org/10.1021/acs.jpca.1c05102>. PMID: 34463510.
- [12] Matthew Frankel, Mario De Florio, Enrico Schiassi, and Lina Sela. Hybrid chemical and data-driven model for stiff chemical kinetics using a physics-informed neural network. *Engineering Proceedings*, 69(1):40, 2024.
- [13] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, mar 2021. ISSN 2522-5839. doi: 10.1038/s42256-021-00302-5. URL <http://dx.doi.org/10.1038/s42256-021-00302-5>.

-
- [14] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.
- [15] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.
- [16] Khemraj Shukla, Juan Diego Toscano, Zhicheng Wang, Zongren Zou, and George Em Karniadakis. A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks. *Computer Methods in Applied Mechanics and Engineering*, 431:117290, 2024. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2024.117290>. URL <https://www.sciencedirect.com/science/article/pii/S0045782524005462>.
- [17] Somdatta Goswami, Ameya D. Jagtap, Hessam Babaei, Bryan T. Susi, and George Em Karniadakis. Learning stiff chemical kinetics using extended deep neural operators. *Computer Methods in Applied Mechanics and Engineering*, 419:116674, 2024. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2023.116674>. URL <https://www.sciencedirect.com/science/article/pii/S0045782523007971>.
- [18] Simone Venturi and Tiernan Casey. SVD perspectives for augmenting DeepONet flexibility and interpretability. *Computer Methods in Applied Mechanics and Engineering*, 403:115718, 2023. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2022.115718>. URL <https://www.sciencedirect.com/science/article/pii/S0045782522006739>.
- [19] Anuj Kumar and Tarek Echekki. Combustion chemistry acceleration with DeepONets. *Fuel*, 365:131212, 2024. ISSN 0016-2361. doi: <https://doi.org/10.1016/j.fuel.2024.131212>. URL <https://www.sciencedirect.com/science/article/pii/S0016236124003582>.
- [20] Yuting Weng, Han Li, Hao Zhang, Zhi X. Chen, and Dezhi Zhou. Extended Fourier Neural Operators to learn stiff chemical kinetics under unseen conditions. *Combustion and Flame*, 272:113847, 2025. ISSN 0010-2180. doi: <https://doi.org/10.1016/j.combustflame.2024.113847>. URL <https://www.sciencedirect.com/science/article/pii/S001021802400556X>.
- [21] Kamaljiyoti Nath, Additi Pandey, Bryan T. Susi, Hessam Babaei, and George Em Karniadakis. AMORE: Adaptive multi-output operator network for stiff chemical kinetics, 2025. URL <https://arxiv.org/abs/2510.12999>.
- [22] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022. URL <https://arxiv.org/abs/2111.00396>.
- [23] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv e-prints*, pages arXiv–2312, 2023.
- [24] Zheyuan Hu, Nazanin Ahmadi Daryakenari, Qianli Shen, Kenji Kawaguchi, and George Em Karniadakis. State-space models are accurate and efficient neural operators for dynamical systems. *arXiv preprint arXiv:2409.03231*, 2024.
- [25] Emilio Vital Brazil, Eduardo Soares, Victor Yukio Shirasuna, Renato Cerqueira, Dmitry Zubarev, and Kristin Schmidt. A Mamba-based foundation model for chemistry. In *AI for Accelerated Materials Design-NeurIPS 2024*, 2024.
- [26] Anri Lombard, Shane Acton, Ulrich Armel Mbou Sob, and Jan Buys. Molecular generation with state space sequence models. In *NeurIPS 2024 Workshop on AI for New Drug Modalities*, 2024. URL <https://openreview.net/forum?id=lib5oyTQIb>.
- [27] Bohao Xu, Yingzhou Lu, Chenhao Li, Ling Yue, Xiao Wang, Nan Hao, Tianfan Fu, and Jim Chen. SMILES-Mamba: Chemical mamba foundation models for drug ADMET prediction. *arXiv preprint arXiv:2408.05696*, 2024.
- [28] Bohao Xu, Yingzhou Lu, Yoshitaka Inoue, Namkyeong Lee, Tianfan Fu, and Jintai Chen. Protein-Mamba: Biological Mamba Models for Protein Function Prediction. *arXiv preprint arXiv:2409.14617*, 2024.
- [29] Zheyuan Hu, Qianying Cao, Kenji Kawaguchi, and George Em Karniadakis. Deepomamba: State-space model for spatio-temporal pde neural operator learning. *J. Comput. Phys.*, 540:114272, 2025. URL <https://api.semanticscholar.org/CorpusID:280547144>.

- [30] David G Goodwin, Harry K Moffat, and Raymond L Speth. Cantera: An Object-oriented Software Toolkit for Chemical Kinetics, Thermodynamics, and Transport Processes. Version 2.3.0. *Zenodo*, 2017. doi: 10.5281/zenodo.170284.
- [31] George Karniadakis and Spencer J Sherwin. *Spectral/hp element methods for computational fluid dynamics*. Oxford University Press, 2005.
- [32] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [33] Gregory P Smith, David M Golden, Michael Frenklach, Nigel W Moriarty, Boris Eiteneer, Mikhail Goldenberg, C Thomas Bowman, Ronald K Hanson, Soonho Song, William C Gardiner Jr, et al. GRI 3.0 mechanism. *Gas Research Institute*, 1999.

Appendices

In the following sections, we shall touch upon several relevant aspects corresponding to the data, implementation and results that we could not include in the main body of the paper.

A Range of Data

In this section, we report the maximum and minimum values from the training and testing dataset. For Syngas A dataset, the maximum and minimum values for both training and testing dataset is tabulated in Table 6, and for Syngas B we divided the dataset into two parts, as mentioned above, and we tabulate the training and testing dataset minimum and maximum values in Table 7. We also have Table 8 tabulating the maximum and minimum values from the GRI dataset.

Table 6: Syngas A dataset values which is generated with equivalence ratios and temperatures from a uniform distribution.

State Variables	Max	Min
Density	7.062e−1	5.239e−2
Temperature	2.984e3	1.000e3
H ₂	2.926e−2	8.978e−6
O ₂	2.153e−1	6.199e−5
O	2.601e−2	1.298e−12
OH	2.848e−2	8.181e−13
H ₂ O	2.497e−1	1.754e−3
H	5.538e−3	7.335e−13
HO ₂	3.289e−4	1.727e−12
CO	3.270e−1	6.264e−4
CO ₂	4.839e−1	2.134e−5
HCO	5.301e−6	0.000
N ₂	7.091e−1	3.260e−1

B Computational Time

In this section, we shall list the amount of time it took to train the Kinetic-Mamba framework on different problems using their respective training dataset that we discussed in this paper, as well as the amount of time it took to make a prediction on the test dataset. We want to mention that the models were trained on an NVIDIA L40S GPU (<https://www.nvidia.com/en-us/data-center/l40s/>) in a network cluster.

Table 7: Syngas B dataset values generated with equivalence ratios and temperatures sampled from a random distribution.

State Variables	Max	Min
Pressure	2.606	1.010
Temperature	3.164e3	5.006e2
H ₂	4.009e-3	7.654e-5
O ₂	1.825e-1	1.365e-2
O	3.202e-2	2.402e-24
OH	1.545e-2	7.039e-27
H ₂ O	2.870e-2	4.609e-24
H	1.424e-3	5.883e-28
HO ₂	1.009e-4	1.106e-22
CO	2.785e-1	3.390e-2
CO ₂	2.555e-1	8.990e-22
HCO	2.297e-6	5.353e-30
N ₂	6.284e-1	5.811e-1

Table 8: GRI-Mech 3.0 dataset values generated with equivalence ratios and temperatures sampled from a uniform distribution.

State Variables	Max	Min
Temperature	2.870e3	1.000e3
H ₂	1.787e-2	5.446e-5
H	3.873e-3	5.110e-14
O	2.356e-2	1.160e-12
O ₂	2.298e-1	6.809e-3
OH	1.921e-2	3.626e-14
H ₂ O	1.250e-1	1.104e-14
HO ₂	1.428e-4	1.681e-12
H ₂ O ₂	9.155e-6	1.595e-17
CO	2.482e-1	4.318e-3
CO ₂	1.871e-1	3.257e-12
HCO	2.748e-6	5.465e-17
CH ₂ O	4.200e-8	9.630e-21
N	2.051e-6	2.698e-27
NH	2.753e-7	1.500e-28
NH ₂	5.226e-8	3.043e-31
NH ₃	3.004e-8	5.108e-33
NNH	1.079e-7	5.047e-19
NO	1.280e-2	7.873e-27
NO ₂	6.751e-6	5.099e-37
HNO	6.917e-7	2.777e-30
HNCO	2.934e-8	2.370e-31
NCO	6.952e-9	5.533e-31
N ₂	6.623e-1	5.411e-1

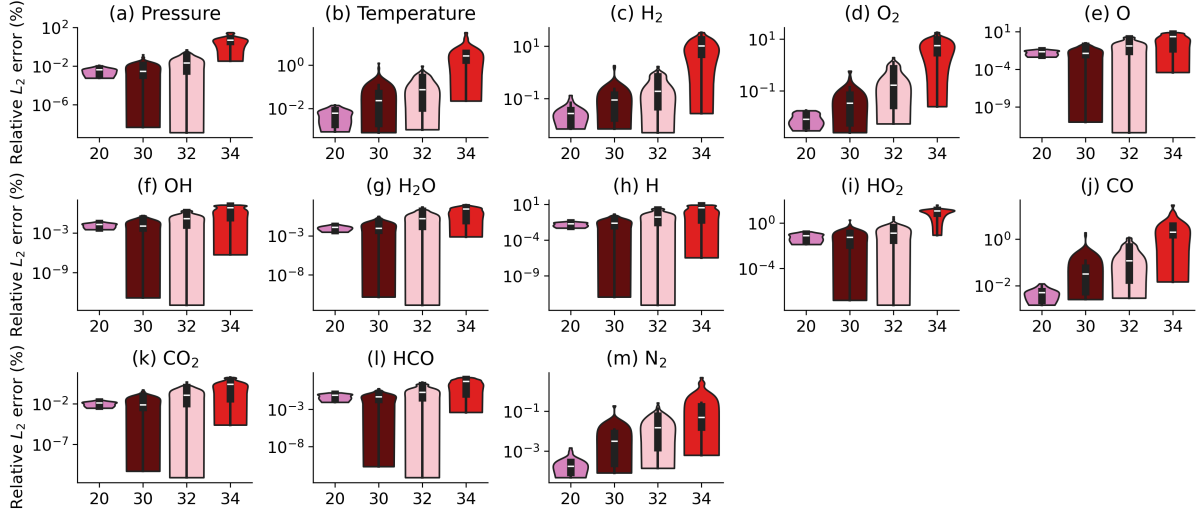


Fig. 19: **Syngas B violin plot using revised error metric during extrapolation:** Relative L_2 error for Syngas B extrapolation dataset when extrapolation was performed using pre-trained model with error values depicted in log scale on y-axis using the clipped denominator error metric.

1. In the problem corresponding to the Syngas A dataset Section 3.1.1, we observe that the total training time is 161,315.47 seconds (44.81 hours) and the prediction time is 2.77 seconds.
2. When we transition to the mass-conserving Kinetic-Mamba framework on the Syngas A dataset Section 3.1.2, we observe that the total training time is 161,677.78 seconds (44.91 hours) and the prediction time is 2.74 seconds. Moreover, it takes 142.72 seconds to go from the m -dimensional manifold to the $m + 1$ -dimensional manifold.
3. When we use the Syngas B dataset with our Kinetic-Mamba framework which takes the ignition regimes into consideration Section 3.1.3, we note that the training time is 12,743.51 seconds (3.54 hours) and 185,055.27 seconds (51.40 hours) corresponding to the data lying below τK and above τK and the prediction times are 0.78 seconds and 4.38 seconds, respectively.
4. When considering the latent Kinetic-Mamba model on the GRI dataset Section 3.1.4, the training time is 45,764.18 seconds (12.71 hours) and the prediction time is 0.92 seconds.

C Additional Results on Extrapolation

We mentioned inflation in the relative L_2 error observed in violin plot Fig. 16 due to small value of the denominator (true value) due to the magnitudes of mass fractions of species. Hence, we clip the denominator to $\max(\text{den}, \epsilon)$, where $\epsilon = 10^{-3} \times \tau$ where τ is the sample mean of the relative L_2 error of true values with respect to time.